# Different types of Sorting Techniques used in Data Structures

in assadullahkhan

AssadUllahKhan

# Sorting

- ***Sorting*** is a process that organizes a collection of data into either ascending or descending order.
- An ***internal sort*** requires that the collection of data fit entirely in the computer's main memory.
- We can use an ***external sort*** when the collection of data cannot fit in the computer's main memory all at once but must reside in secondary storage such as on a disk.
- We will analyze only internal sorting algorithms.
- Any significant amount of computer output is generally arranged in some sorted order so that it can be interpreted.
- Sorting also has indirect uses. An initial sort of the data can significantly enhance the performance of an algorithm.
- Majority of programming projects use a sort somewhere, and in many cases, the sorting cost determines the running time.
- A comparison-based sorting algorithm makes ordering decisions only on the basis of comparisons.

# Sorting: Definition

- ***Sorting: an operation that segregates items into order form according to specified criterion.***

$$A = \{ 3\ 1\ 6\ 2\ 1\ 3\ 4\ 5\ 9\ 0 \}$$

$$A = \{ 0\ 1\ 1\ 2\ 3\ 3\ 4\ 5\ 6\ 9 \}$$

# Sorting

- Sorting = ordering.
- Sorted = ordered based on a particular way.
- Generally, collections of data are presented in a sorted manner.
- Examples of Sorting:
  - Words in a dictionary are sorted (and case distinctions are ignored).
  - Files in a directory are often listed in sorted order.
  - The index of a book is sorted (and case distinctions are ignored).

# Sorting: Cont'd

- Many banks provide statements that list checks in increasing order (by check number).
- In a newspaper, the calendar of events in a schedule is generally sorted by date.

- Why?
  - Imagine finding the phone number of your friend in your mobile phone, but the phone book is not sorted.

# Review of Complexity

Most of the primary sorting algorithms run on different space and time complexity.

Time Complexity is defined to be the time the computer takes to run a program (or algorithm in our case).

Space complexity is defined to be the amount of memory the computer needs to run a program.

# Complexity (cont.)

Complexity in general, measures the algorithms efficiency in internal factors such as the time needed to run an algorithm.

External Factors (not related to complexity):

- Size of the input of the algorithm
- Speed of the Computer
- Quality of the Compiler

# Types of Sorting Algorithms

There are many, many different types of sorting algorithms, but the primary ones are:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

# Bubble Sort: Idea

- Idea: bubble in water.
  - Bubble in water moves upward. Why?
- How?
  - When a bubble moves upward, the water from above will move downward to fill in the space left by the bubble.

# Bubble Sort

- The list is divided into two sublists: sorted and unsorted.
- The largest element is bubbled from the unsorted list and moved to the sorted sublist.
- After that, the wall moves one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones.
- Each time an element moves from the unsorted part to the sorted part one sort pass is completed.
- Given a list of n elements, bubble sort requires up to n-1 passes to sort the data.

# Bubble Sort Example

9, 6, 2, 12, 11, 9, 3, 7

Bubblesort compares the numbers in pairs from left to right exchanging when necessary. Here the first number is compared to the second and as it is larger they are exchanged.

Now the next pair of numbers are compared. Again the 9 is the larger and so this pair is also exchanged.

In the third comparison, the 9 is not larger than the 12 so no exchange is made. We move on to compare the next pair without any change to the list.

The 12 is larger than the 11 so they are exchanged.

The twelve is greater than the 9 so they are exchanged

The end of the list has been reached so this is the end of the first pass. The twelve at the end of the list must be largest number in the list and so is now in the correct position. We now start a new pass from left to right.

The 12 is greater than the 7 so they are exchanged.

# Bubble Sort Example

First Pass

6, 2, 9, 11, 9, 3, 7, | 12

Second Pass

2, 6, 9, 9, 3, 7, 11, 12

**Notice that this time we do not have to compare the last two numbers as we know the 12 is in position. This pass therefore only requires 6 comparisons.**

# Bubble Sort Example

First Pass

6, 2, 9, 11, 9, 3, 7, 12

Second Pass

2, 6, 9, 9, 3, 7, 11, 12

Third Pass

2, 6, 9, 3, 7, 9, 11, 12

**This time the 11 and 12 are in position. This pass therefore only requires 5 comparisons.**

# Bubble Sort Example

First Pass

6, 2, 9, 11, 9, 3, 7, 12

Second Pass

2, 6, 9, 9, 3, 7, 11, 12

Third Pass

2, 6, 9, 3, 7, 9, 11, 12

Fourth Pass

2, 6, 3, 7, 9, 9, 11, 12

**Each pass requires fewer comparisons. This time only 4 are needed.**

# Bubble Sort Example

First Pass

6, 2, 9, 11, 9, 3, 7, 12

Second Pass

2, 6, 9, 9, 3, 7, 11, 12

Third Pass

2, 6, 9, 3, 7, 9, 11, 12

Fourth Pass

2, 6, 3, 7, 9, 9, 11, 12

Fifth Pass

2, 3, 6, 7, 9, 9, 11, 12

**The list is now sorted but the algorithm does not know this until it completes a pass with no exchanges.**

# Bubble Sort Example

First Pass

6, 2, 9, 11, 9, 3, 7, 12

Second Pass

2, 6, 9, 9, 3, 7, 11, 12

Third Pass

2, 6, 9, 3, 7, 9, 11, 12

Fourth Pass

2, 6, 3, 7, 9, 9, 11, 12

Fifth Pa

**This pass no exchanges are made so the algorithm knows the list is sorted. It can therefore save time by not doing the final pass. With other lists this check could save much more work.**

Sixth Pass

2, 3, 6, 7, 9, 9, 11, 12

# Bubble Sort Example

## Quiz Time

1. Which number is definitely in its correct position at the end of the first pass?

Answer: The last number must be the largest.

2. How does the number of comparisons required change as the pass number increases?

Answer: Each pass requires one fewer comparison than the last.

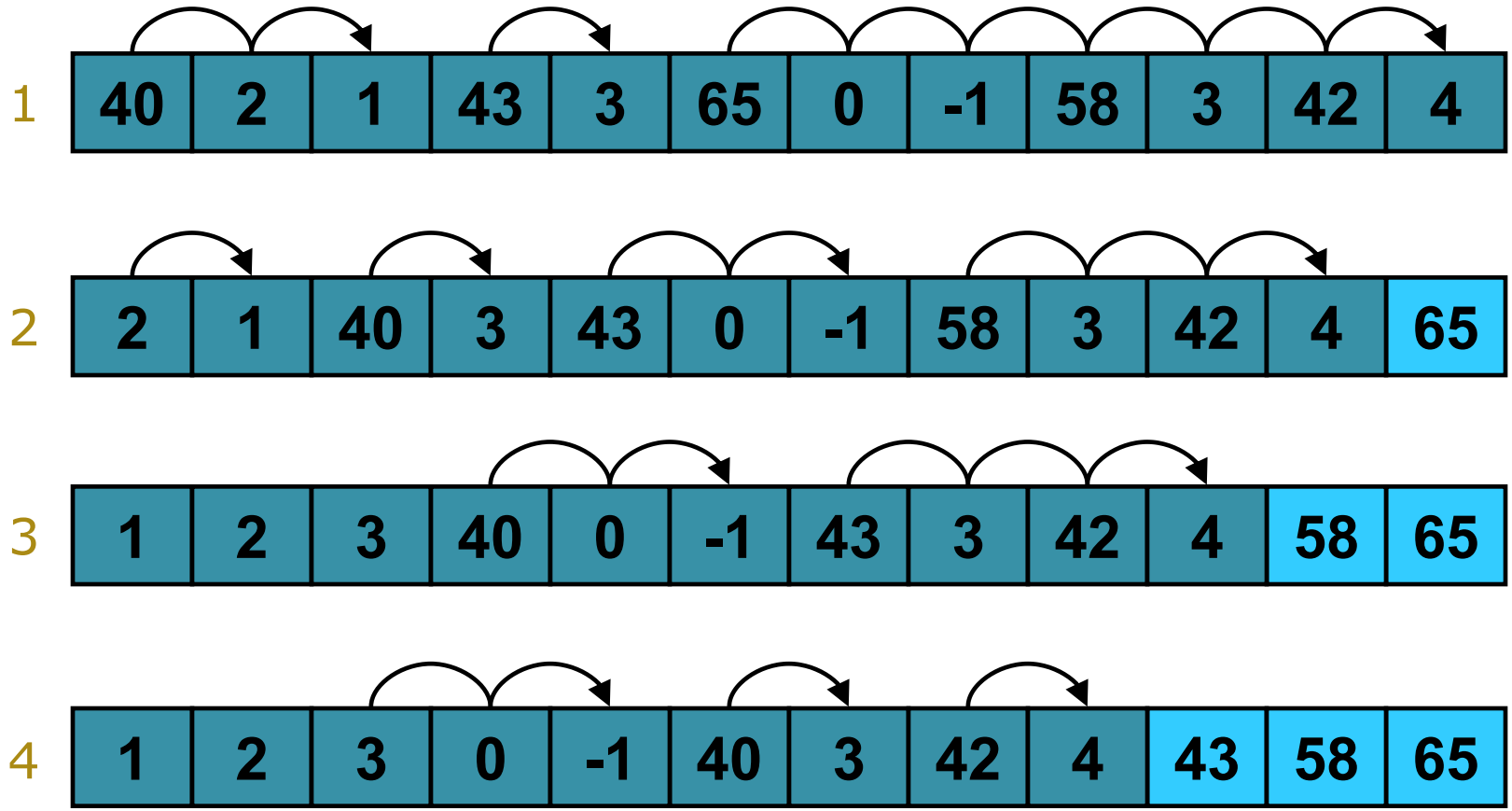
3. How does the algorithm know when the list is sorted?

Answer: When a pass with no exchanges occurs.

4. What is the maximum number of comparisons required for a list of 10 numbers?

Answer: 9 comparisons, then 8, 7, 6, 5, 4, 3, 2, 1 so total 45

# Bubble Sort: Example




- Notice that at least one element will be in the correct position each iteration.

# Bubble Sort: Example

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 5 | 1 | 2 | 0 | -1 | 3 | 3 | 40 | 4 | 42 | 43 | 58 | 65 |
| 6 | 1 | 0 | -1 | 2 | 3 | 3 | 4 | 40 | 42 | 43 | 58 | 65 |
| 7 | 0 | -1 | 1 | 2 | 3 | 3 | 4 | 40 | 42 | 43 | 58 | 65 |
| 8 | -1 | 0 | 1 | 2 | 3 | 3 | 4 | 40 | 42 | 43 | 58 | 65 |

# Selection Sort: Idea

1. We have two group of items:
   - sorted group, and
   - unsorted group
2. Initially, all items are in the unsorted group. The sorted group is empty.
   - We assume that items in the unsorted group unsorted.
   - We have to keep items in the sorted group sorted.

# Selection Sort: Cont'd

1. Select the "best" (eg. Largest) item from the unsorted group, then put the "best" item at the end of the sorted group.
2. Repeat the process until the unsorted group becomes empty.

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 6 | 2 |

↑
**Largest**

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 5 | 1 | 3 | 4 | 2 | 6 |

↑
**Largest**

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

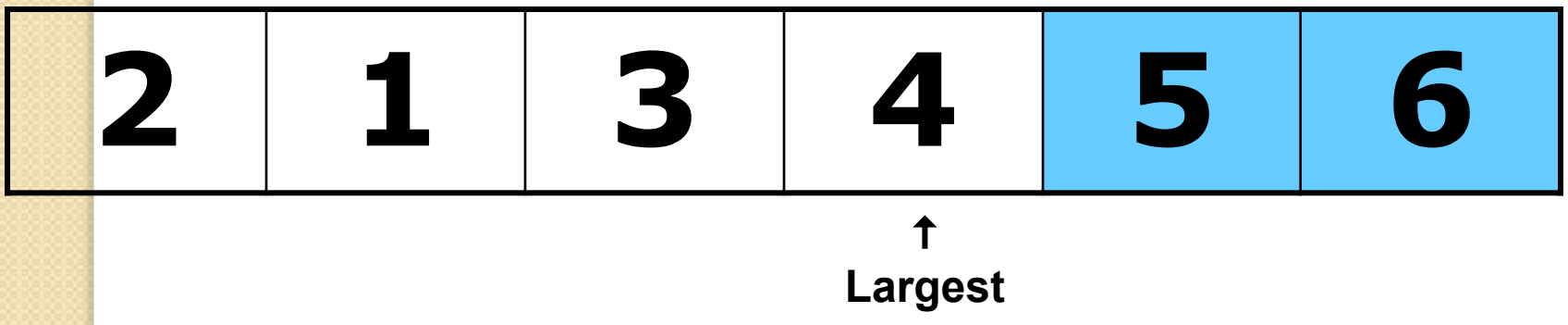
| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort




Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

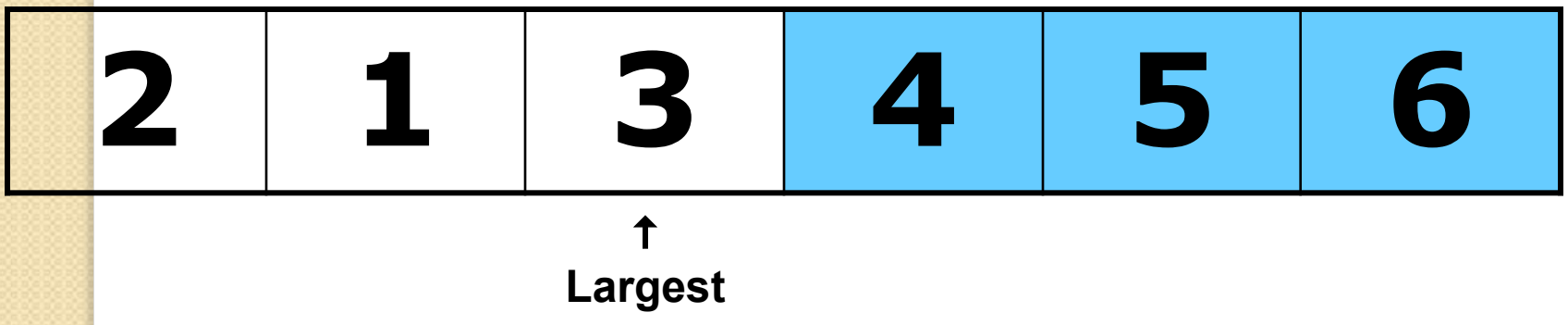
Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort




Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 2 | 1 | 3 | 4 | 5 | 6 |

↑
**Largest**

Comparison

Data Movement

Sorted

# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 |

- Comparison
- Data Movement
- Sorted

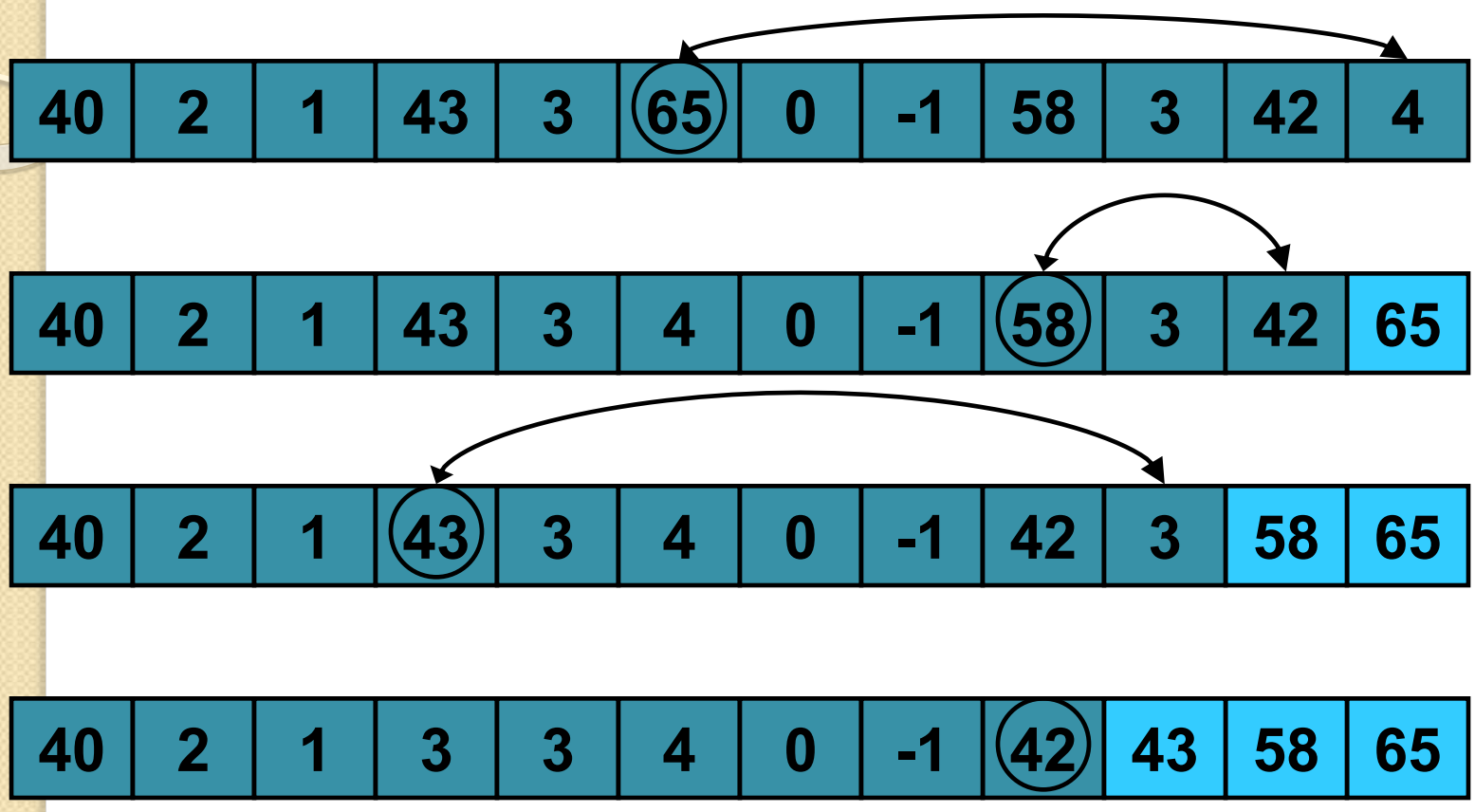
# Selection Sort

| | | | | | |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 |

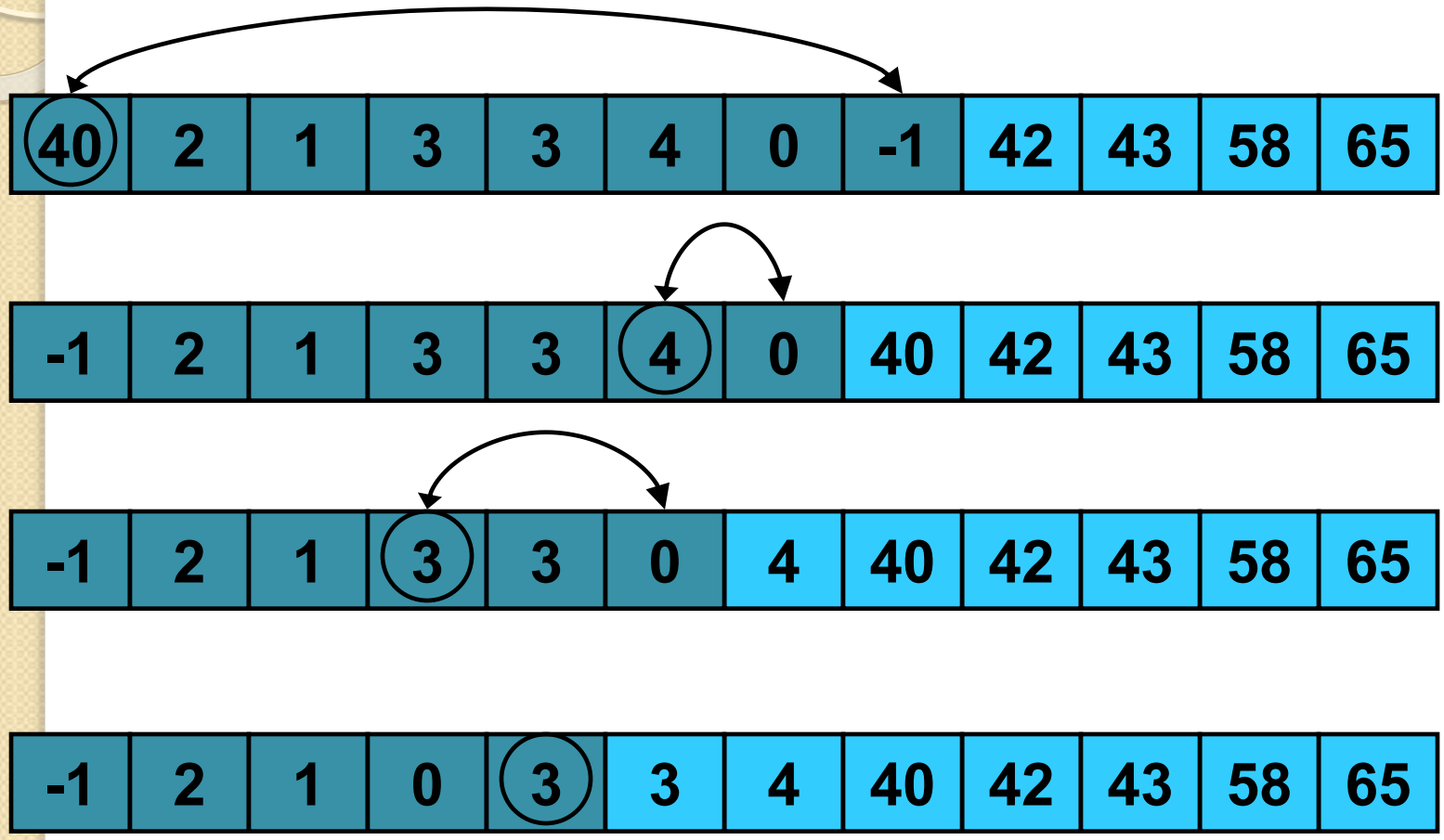
**DONE!**

- Comparison
- Data Movement
- Sorted

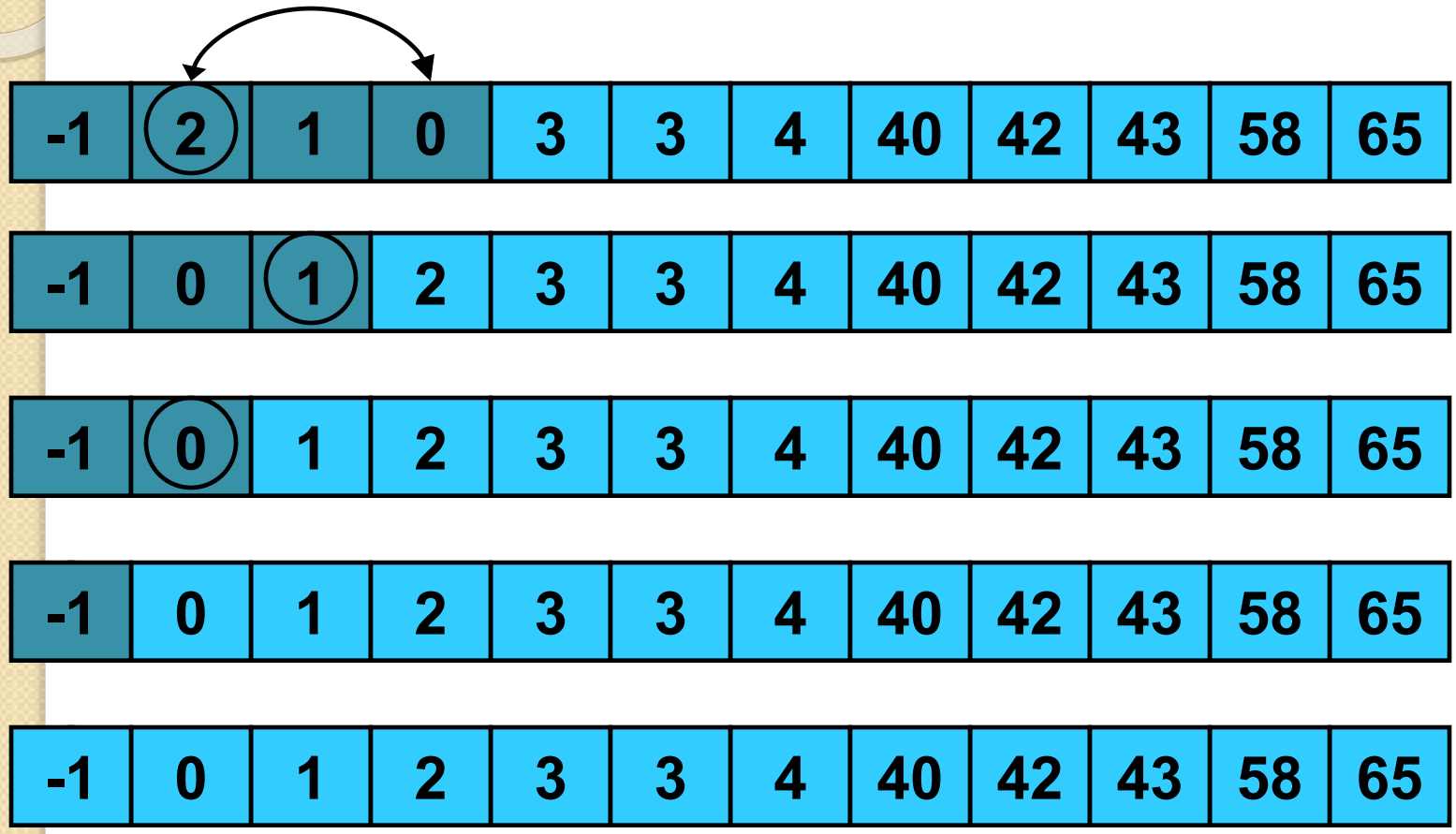
# Selection Sort: Example



# Selection Sort: Example



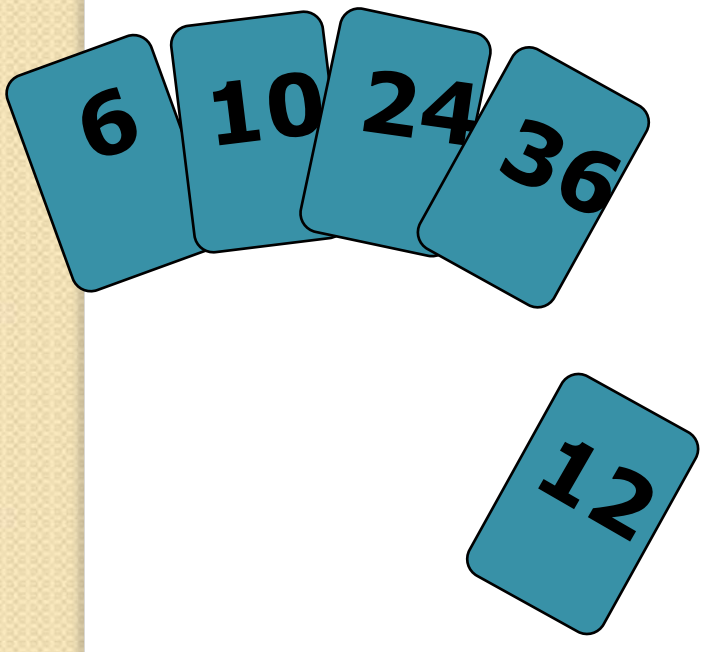
# Selection Sort: Example



# Insertion Sort

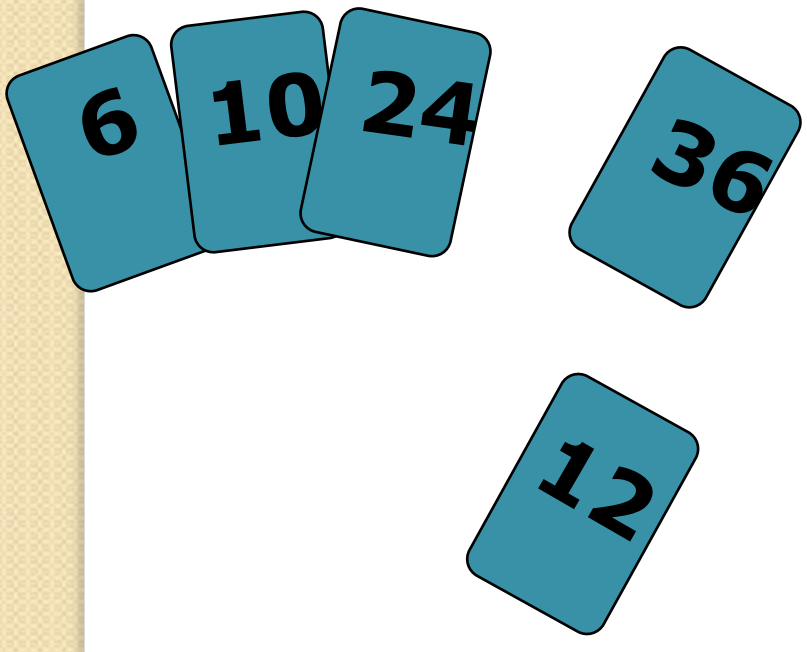
- Idea: like sorting a hand of playing cards
  - Start with an empty left hand and the cards facing down on the table.
  - Remove one card at a time from the table, and insert it into the correct position in the left hand
    - compare it with each of the cards already in the hand, from right to left
  - The cards held in the left hand are sorted
    - these cards were originally the top cards of the pile on the table

# Insertion Sort

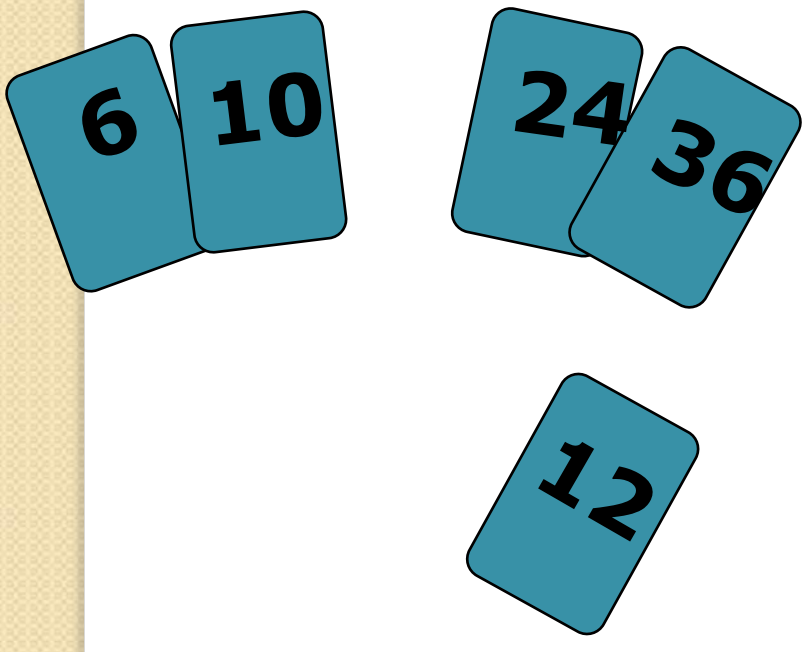



**To insert 12, we need to make room for it by moving first 36 and then 24.**

# Insertion Sort



# Insertion Sort



# Insertion Sort: Idea

- Idea: sorting cards.
  - 8 | 5 9 2 6 3
  - 5 8 | 9 2 6 3
  - 5 8 9 | 2 6 3
  - 2 5 8 9 | 6 3
  - 2 5 6 8 9 | 3
  - 2 3 5 6 8 9 |

# Insertion Sort

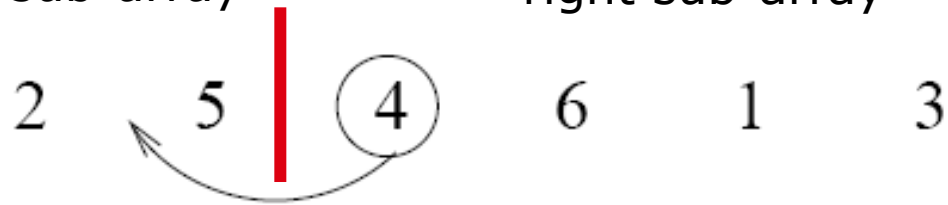
input array

5 2 4 6 1
3

at each iteration, the array is divided in two sub-arrays:

left sub-array

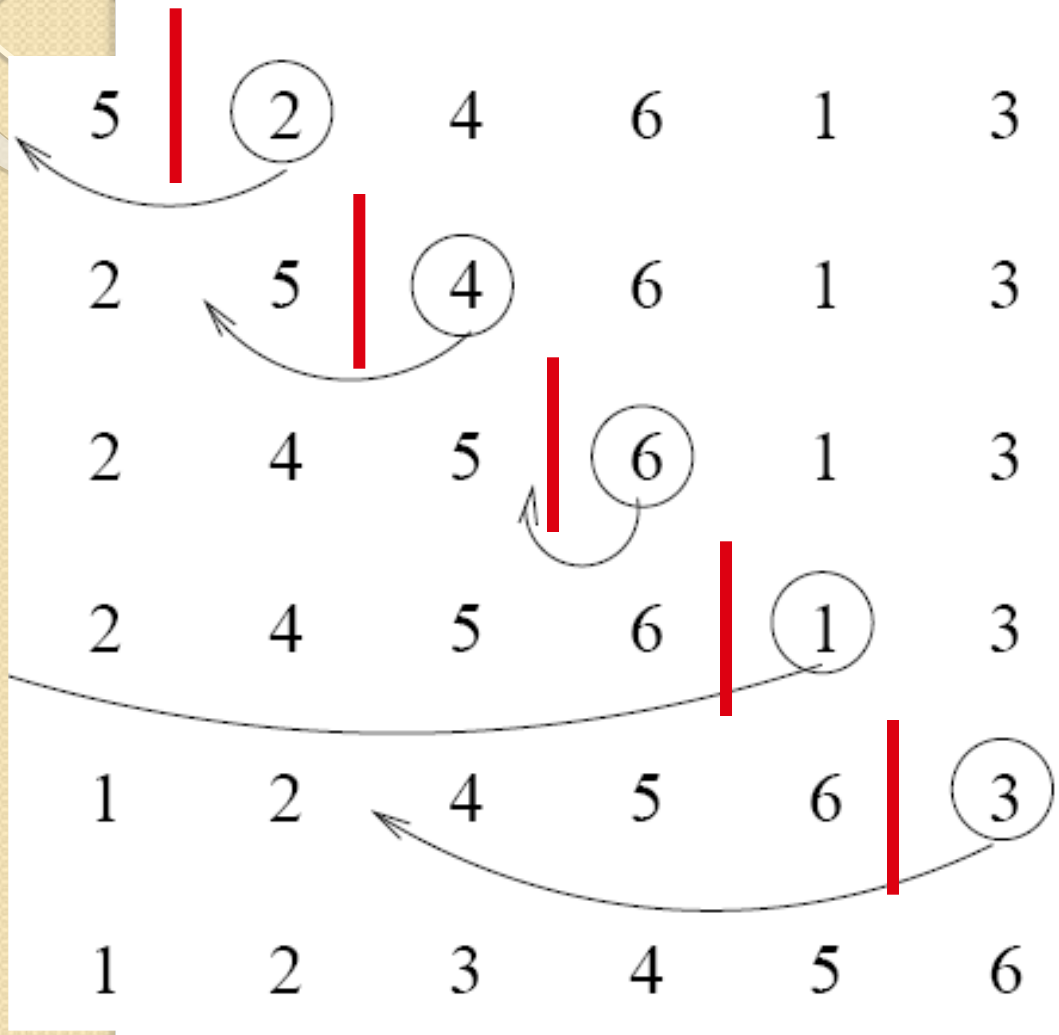
right sub-array

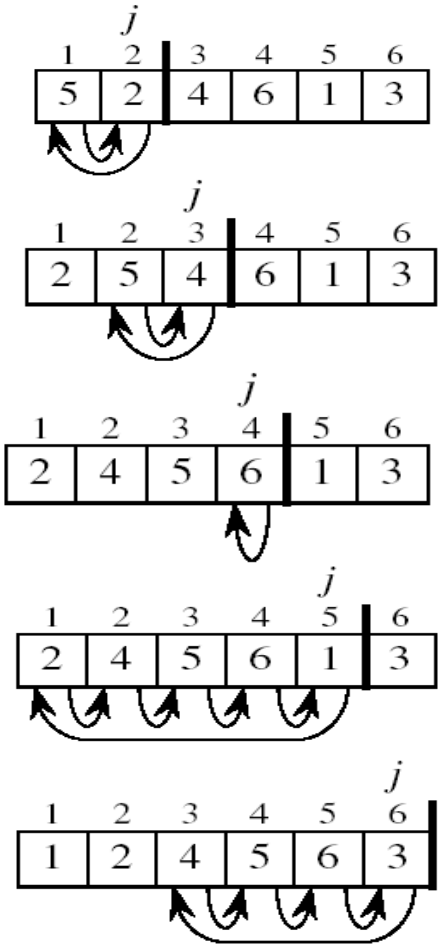



sorted

unsorted

# Insertion Sort

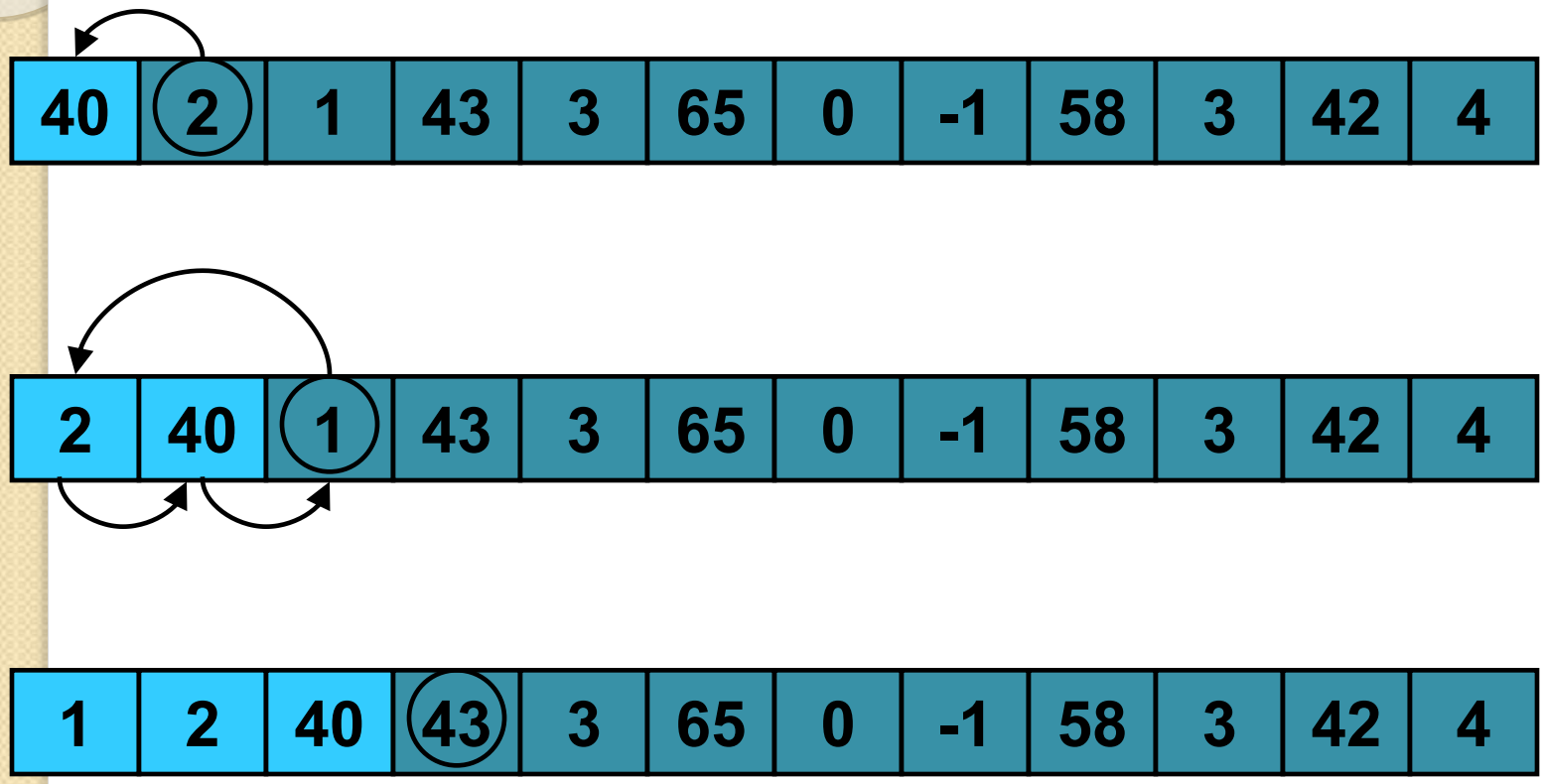


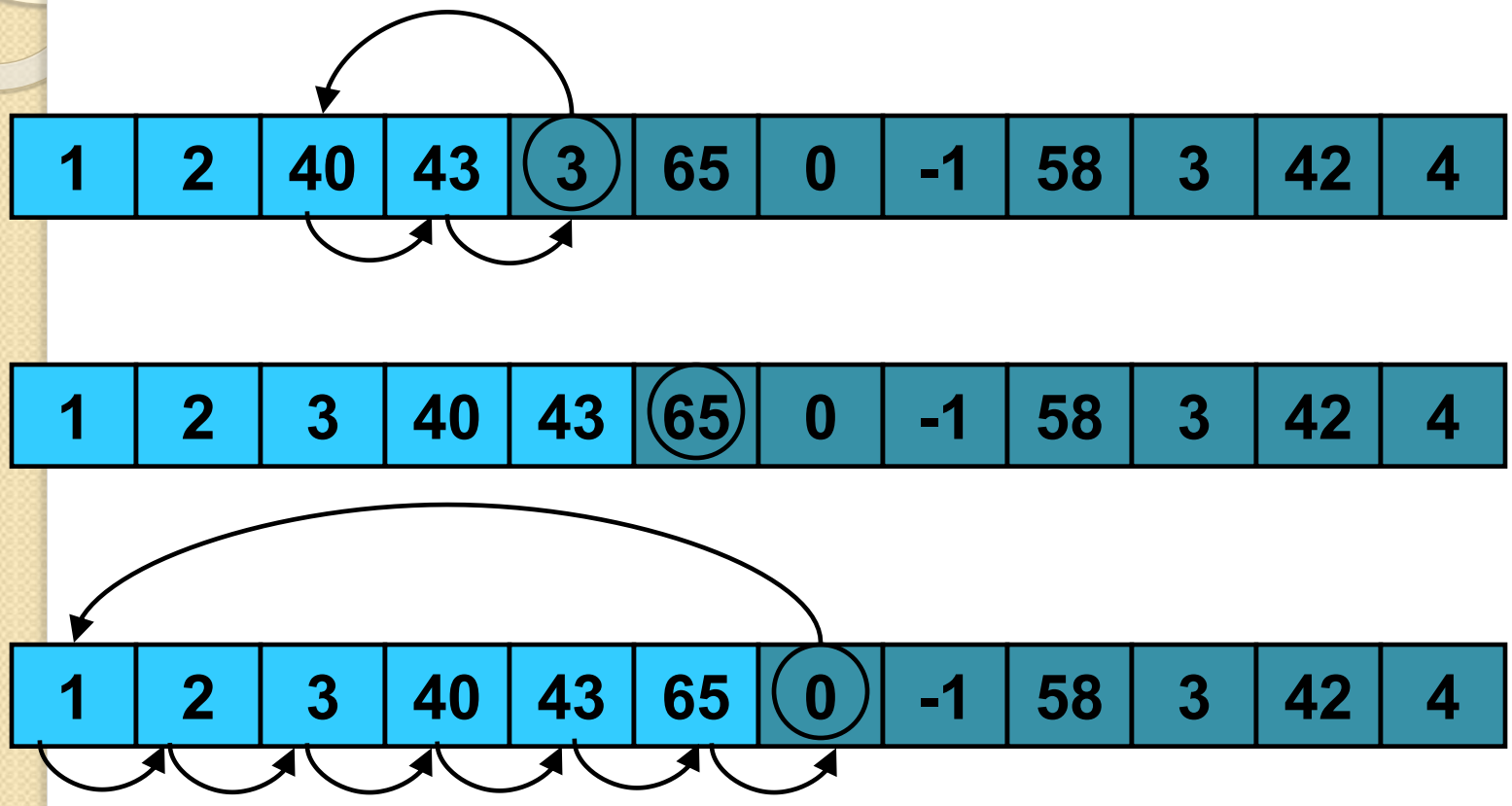
# Insertion Sort: Idea

1. We have two group of items:
   - sorted group, and
   - unsorted group
2. Initially, all items in the unsorted group and the sorted group is empty.
   - We assume that items in the unsorted group unsorted.
   - We have to keep items in the sorted group sorted.
3. Pick any item from, then insert the item at the right position in the sorted group to maintain sorted property.
4. Repeat the process until the unsorted group becomes empty.

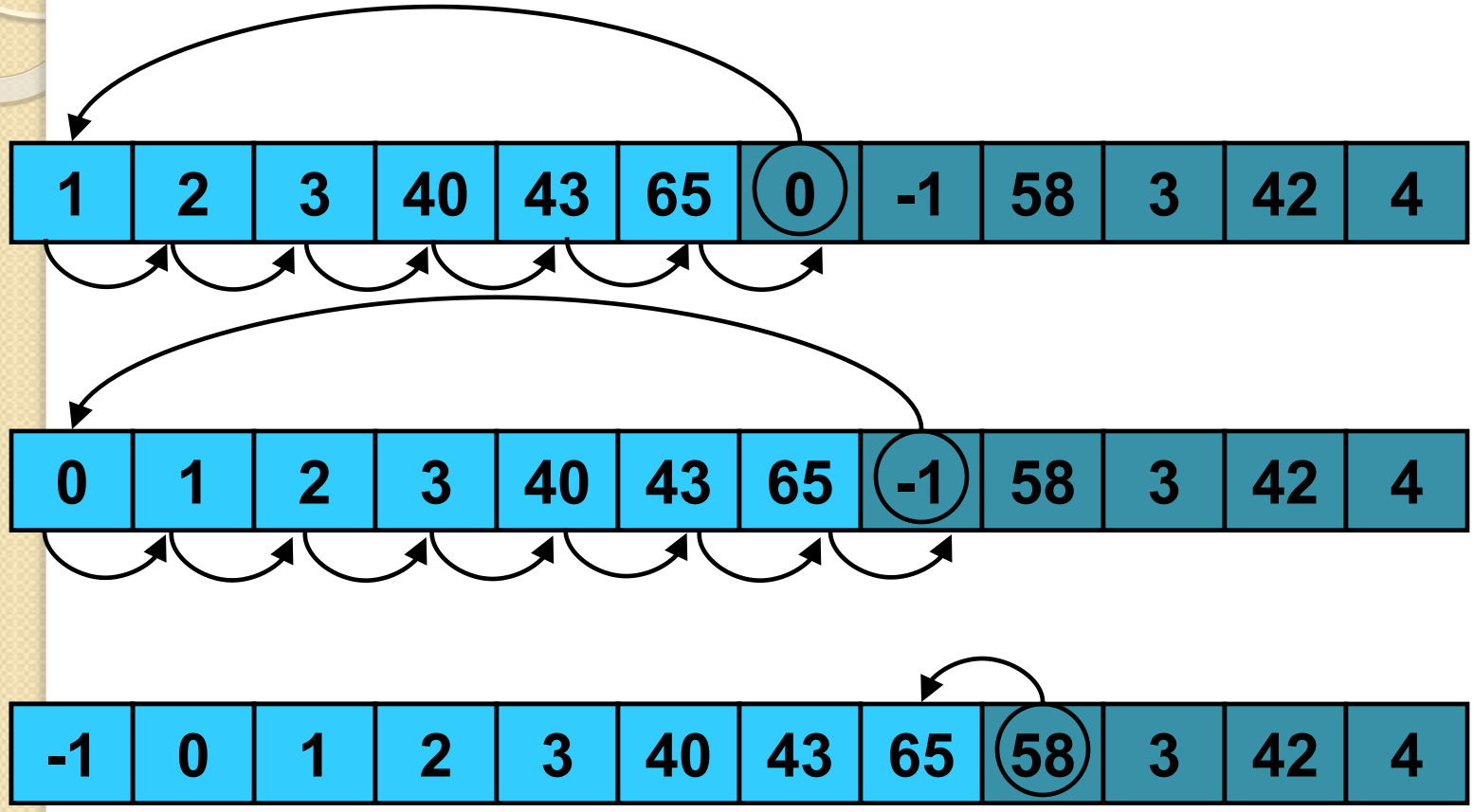
# Insertion Sort: Example



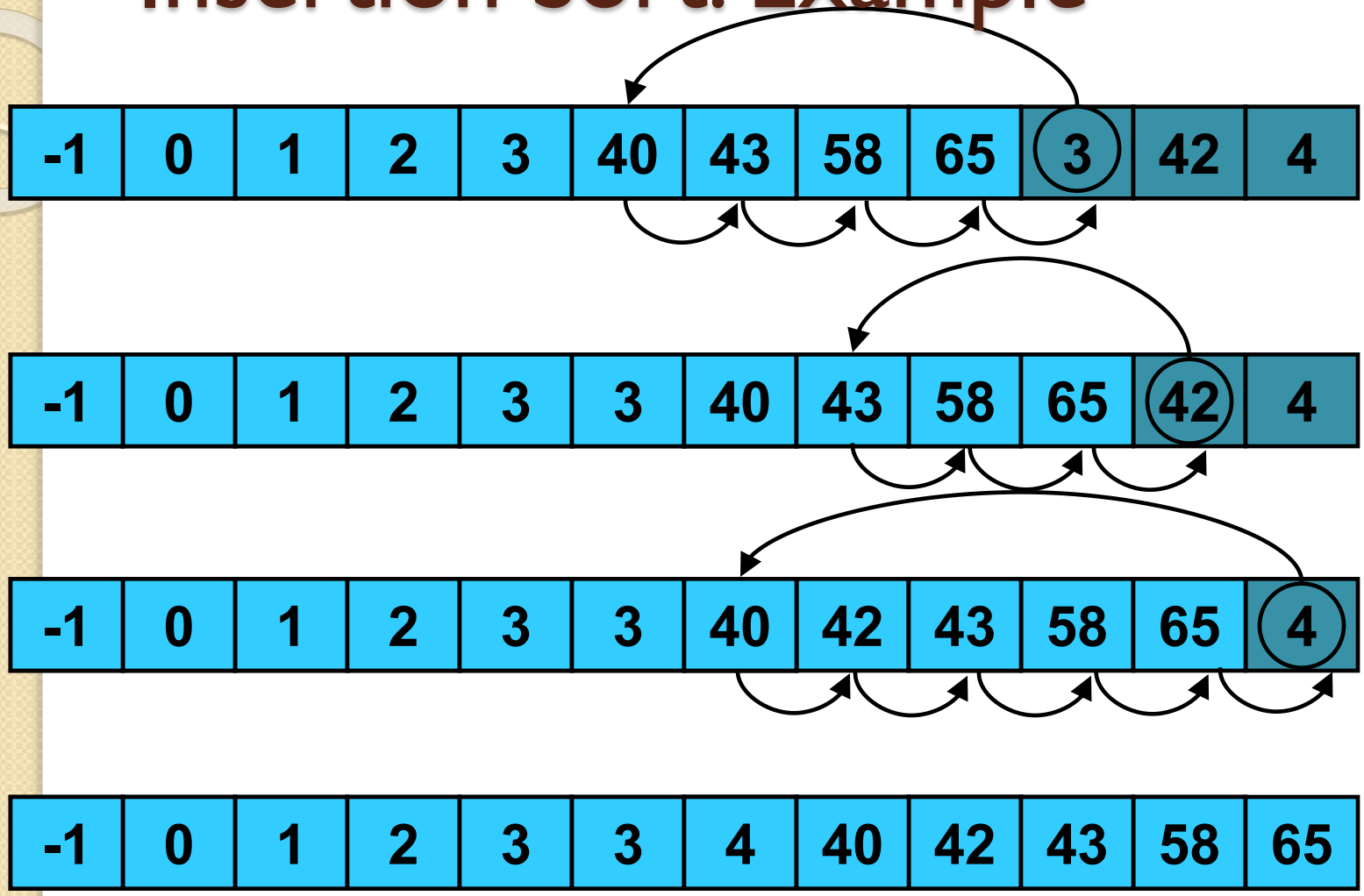
# Insertion Sort: Example



# Insertion Sort: Example



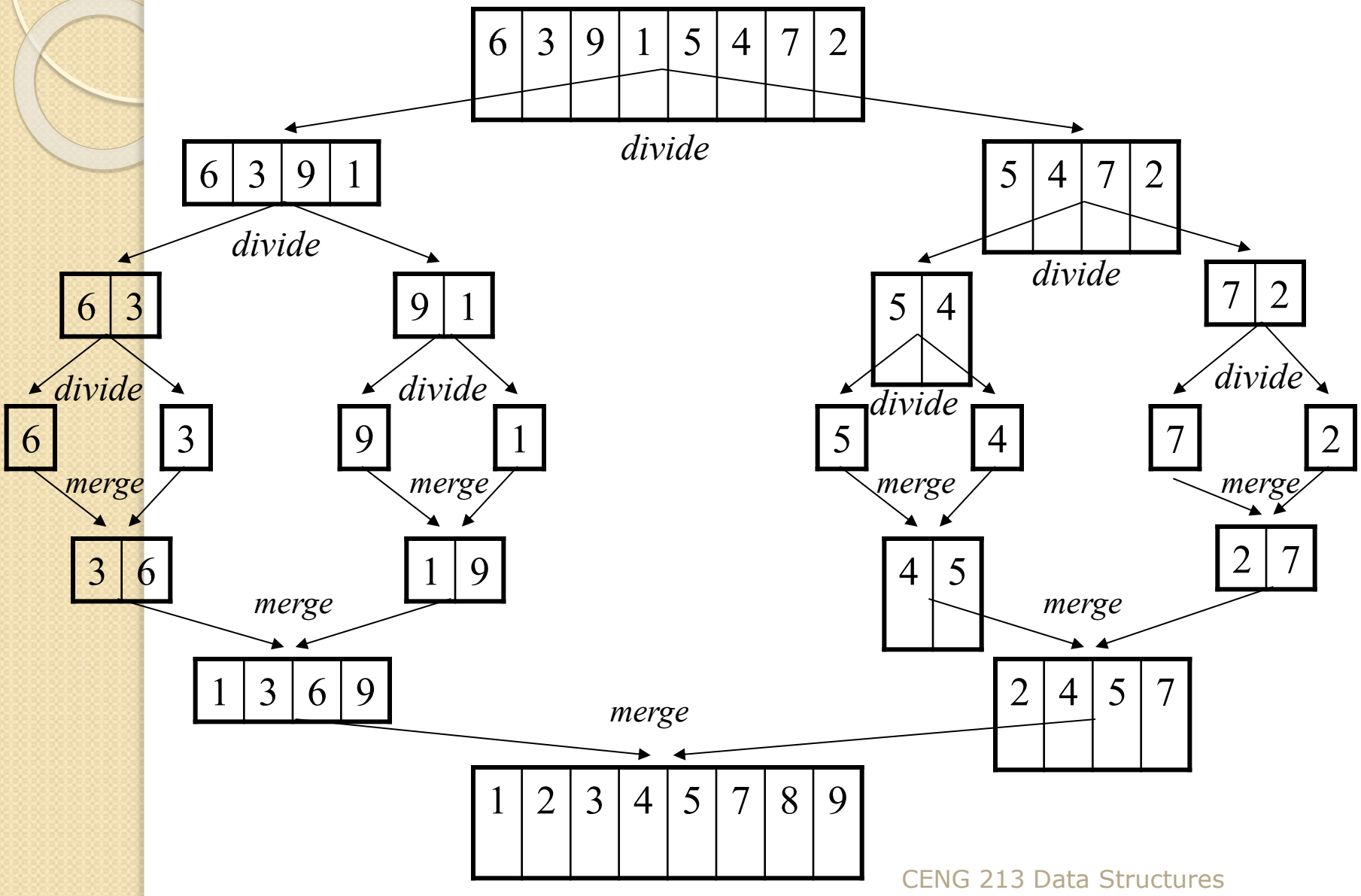
# Insertion Sort: Example



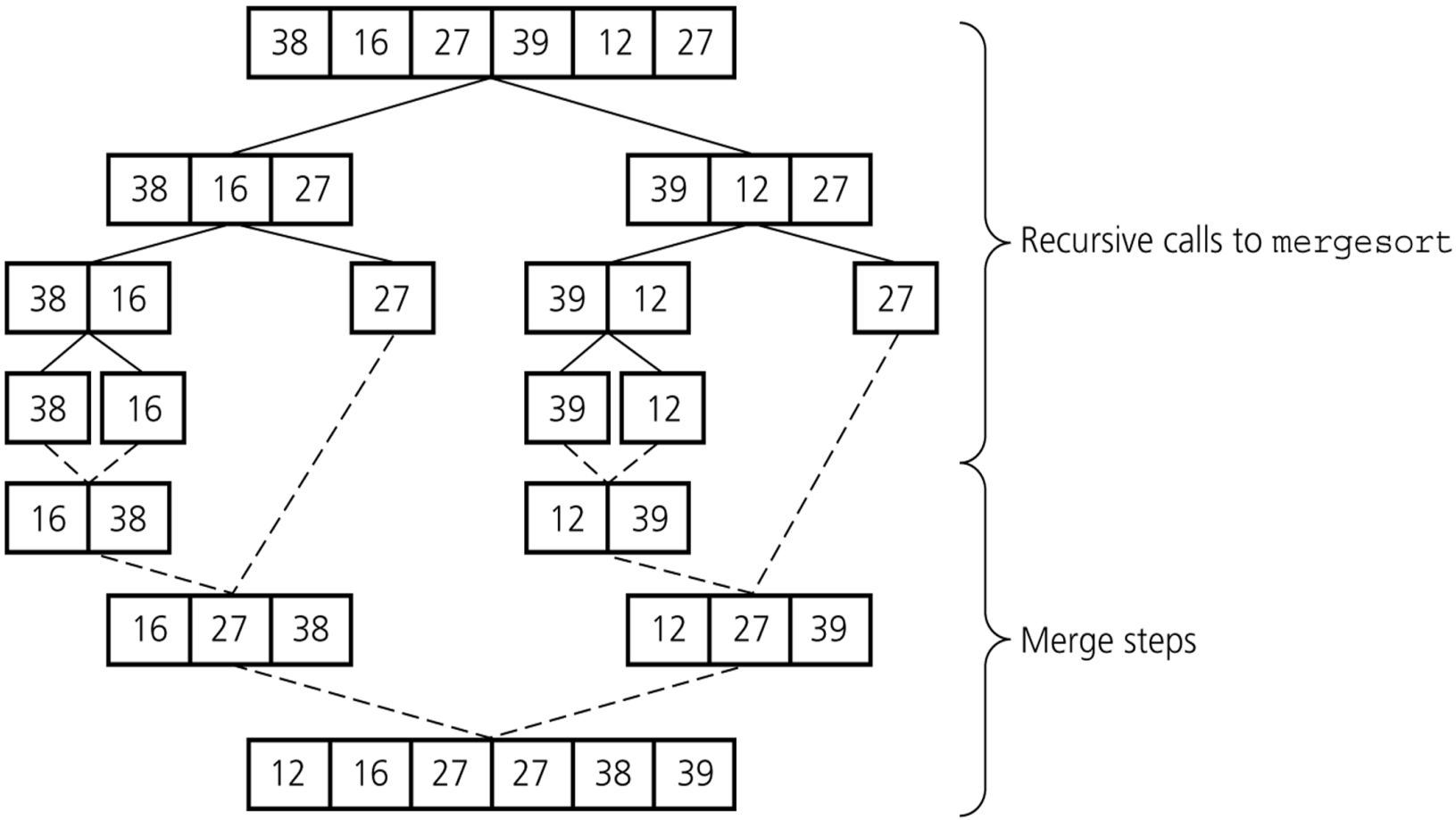
# Merge sort

- Merge sort algorithm is one of two important divide-and-conquer sorting algorithms (the other one is quicksort).
- It is a recursive algorithm.
  - Divides the list into halves,
  - Sort each halve separately, and
  - Then merge the sorted halves into one sorted array.

# Merge sort – Example 1

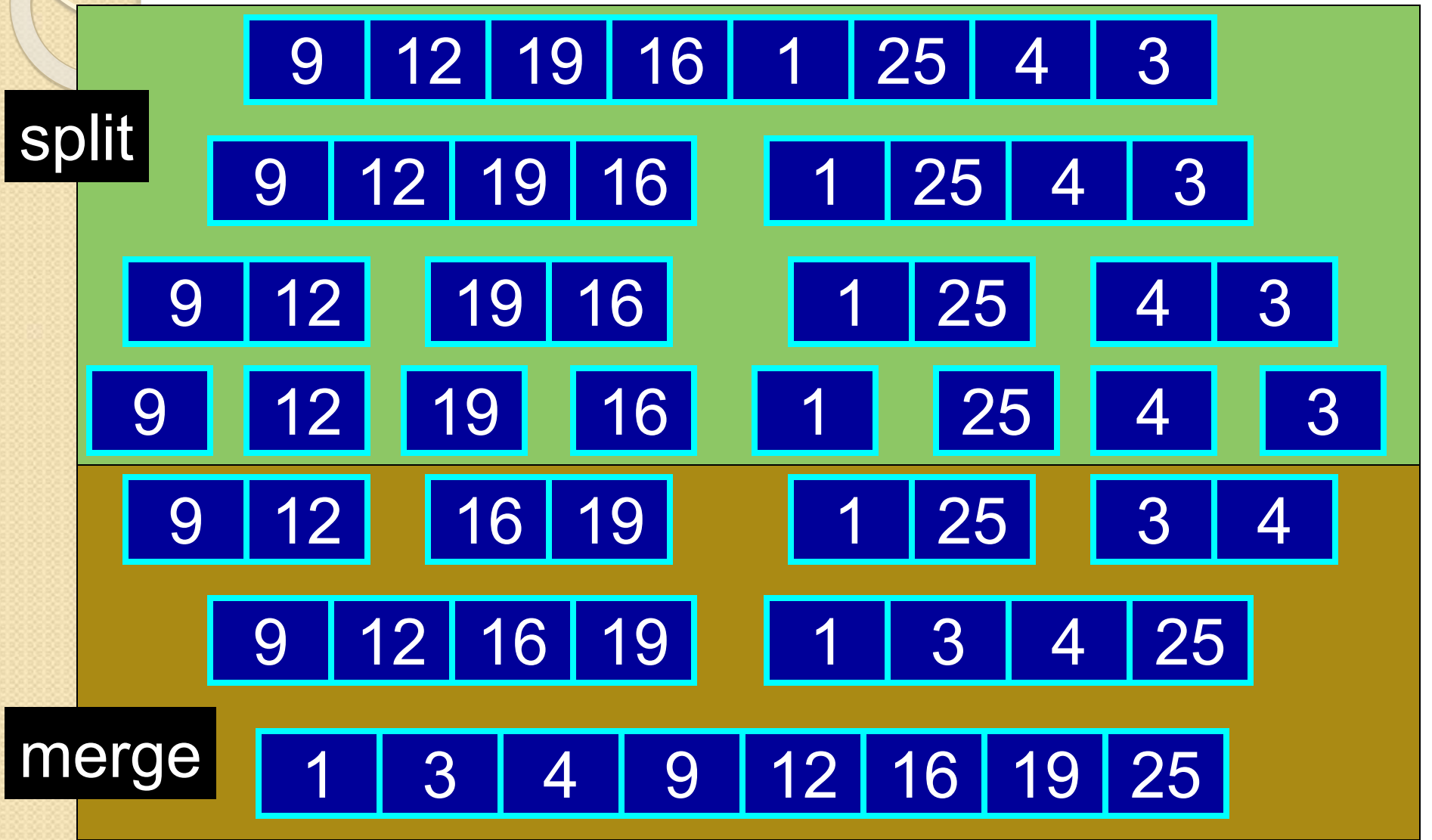


# Mergesort – Example2



# Merge Sort: Example

- Divide and Conquer: Merge Sort



# Notes on Quicksort

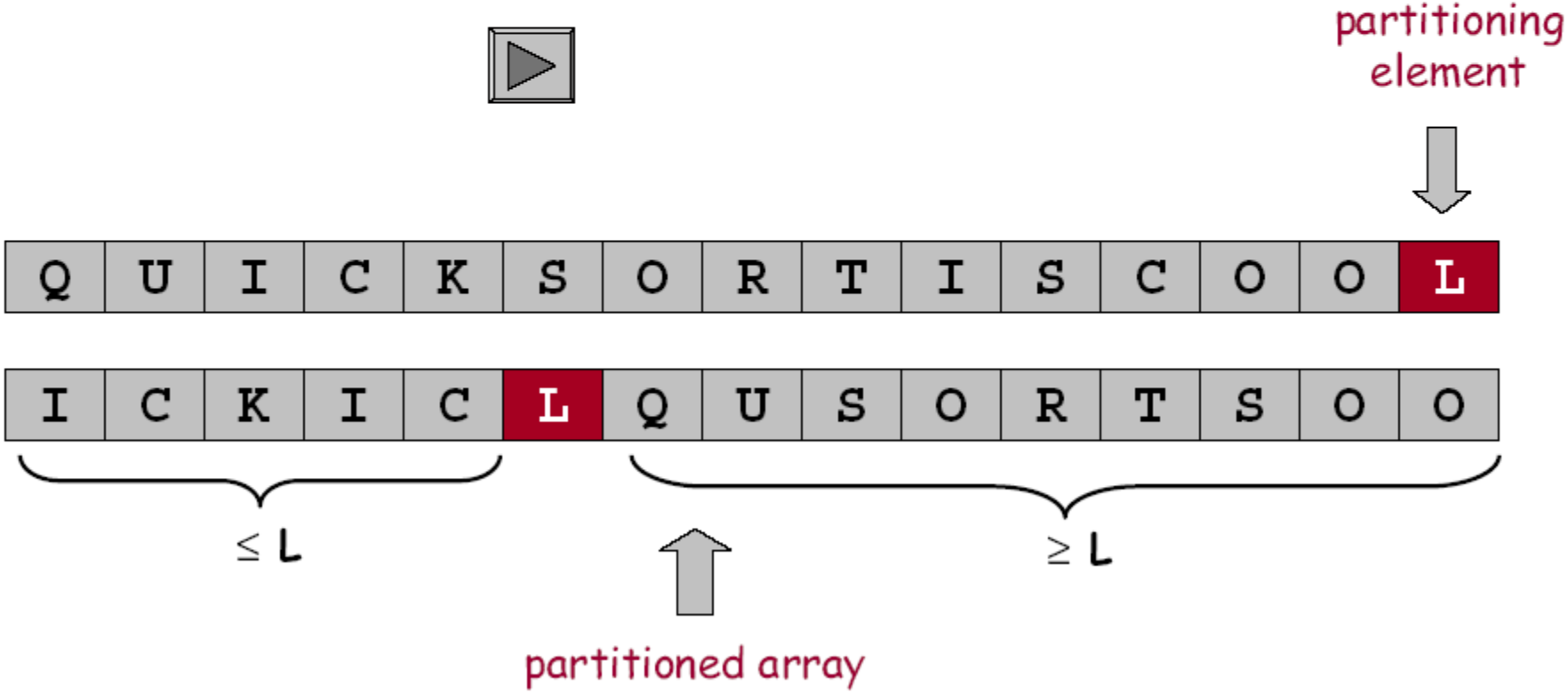
- Quicksort is more widely used than any other sort.
- Quicksort is well-studied, not difficult to implement, works well on a variety of data, and consumes fewer resources that other sorts in nearly all situations.
- Quicksort is O(n*log n) time, and O(log n) additional space due to recursion.

# Quicksort

C. A. R. Hoare

Quicksort.

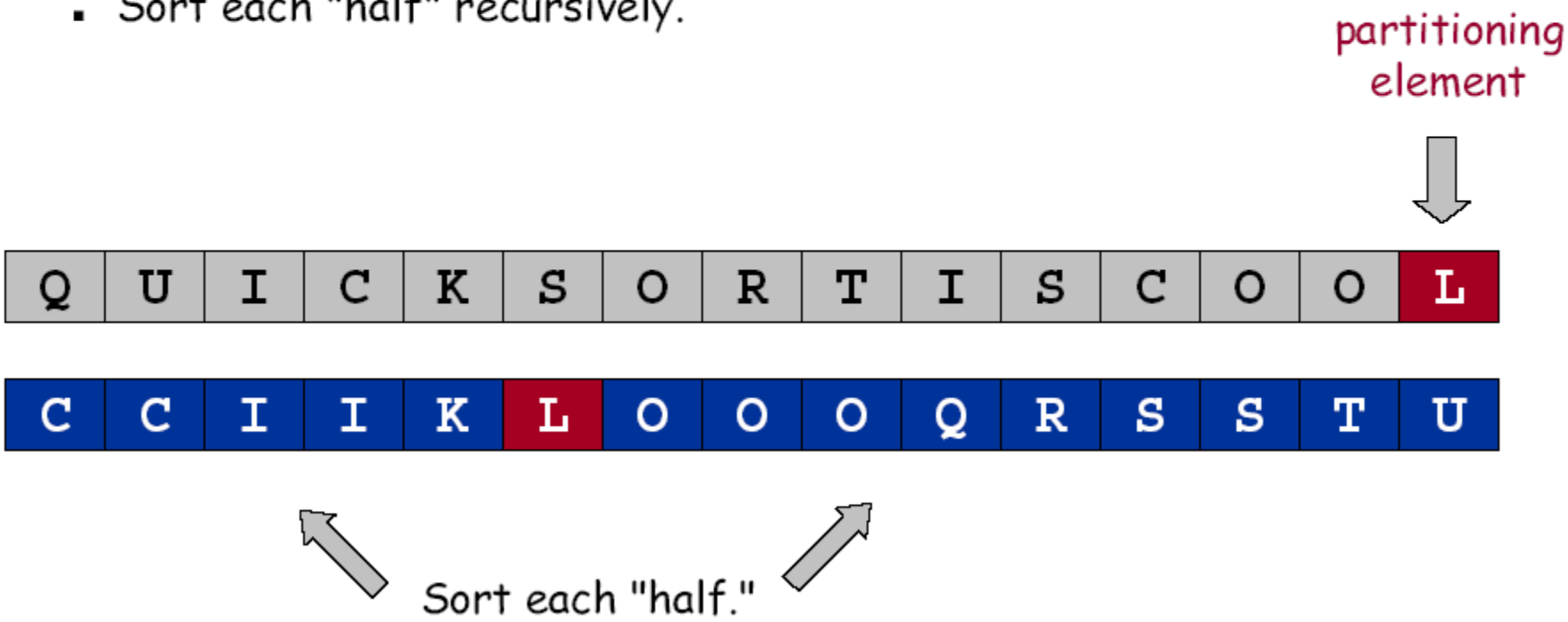
- Partition array so that:
  - some partitioning element `a[m]` is in its final position
  - no larger element to the left of `m`
  - no smaller element to the right of `m`



# Quicksort

Quicksort.

- Partition array so that:
  - some partitioning element `a[m]` is in its final position
  - no larger element to the left of `m`
  - no smaller element to the right of `m`
- Sort each "half" recursively.



# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Q | U | I | C | K | S | O | R | T | I | S | C | O | O | L |

partition element

unpartitioned

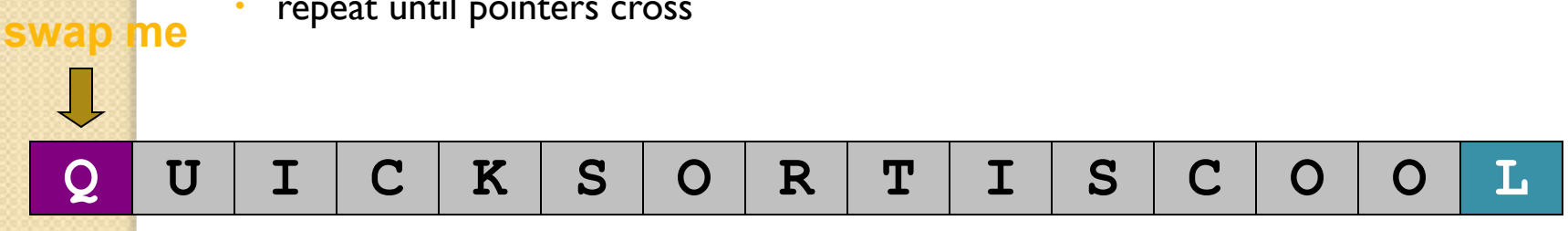
left

partitioned

right

# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross




partition element

unpartitioned

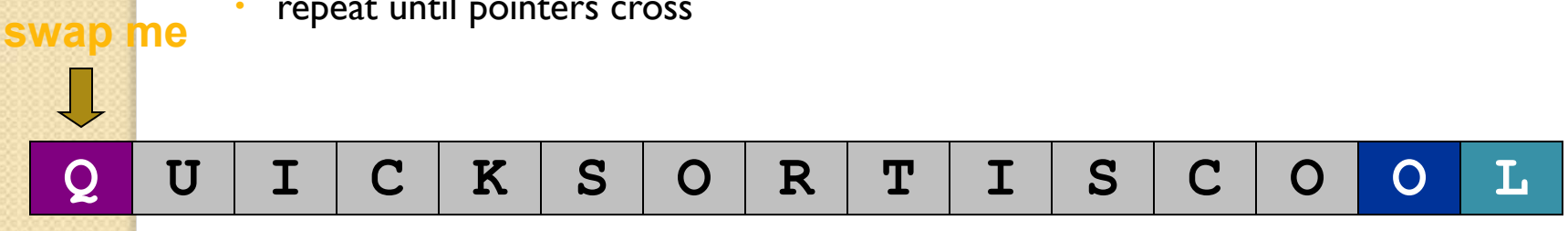
left

partitioned

right

# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross




partition element

unpartitioned

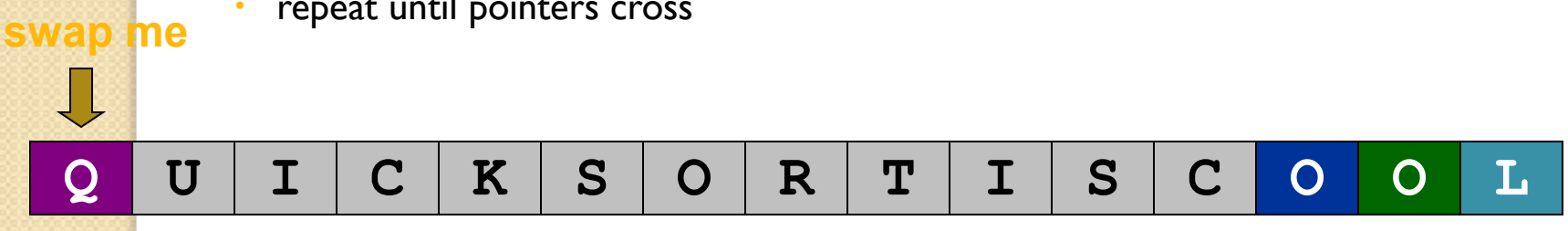
left

partitioned

right

# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross




partition element

unpartitioned

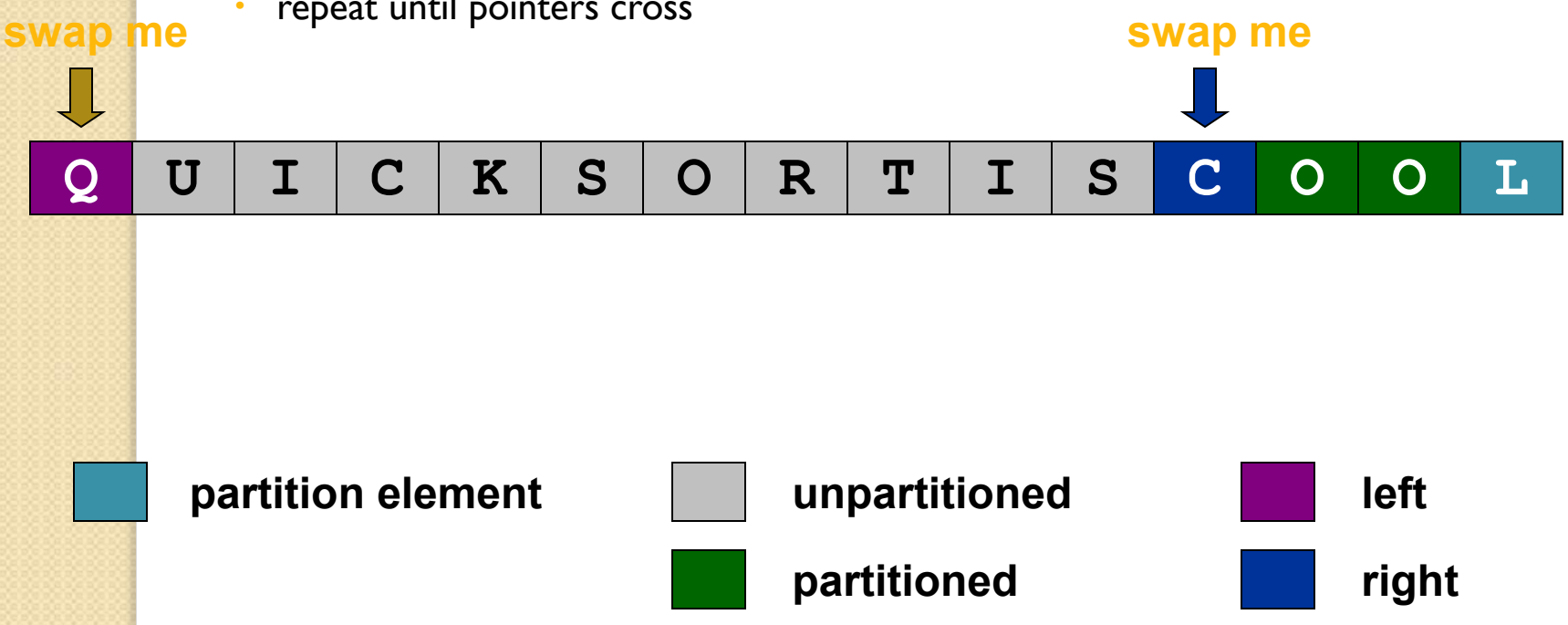
left

partitioned

right

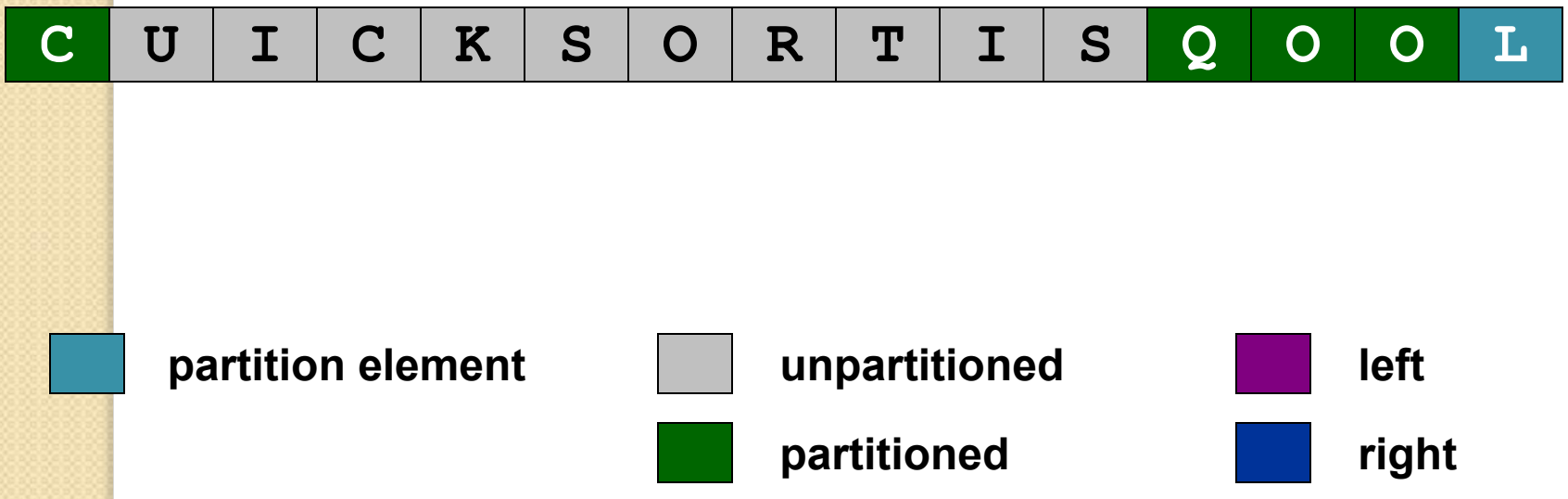
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



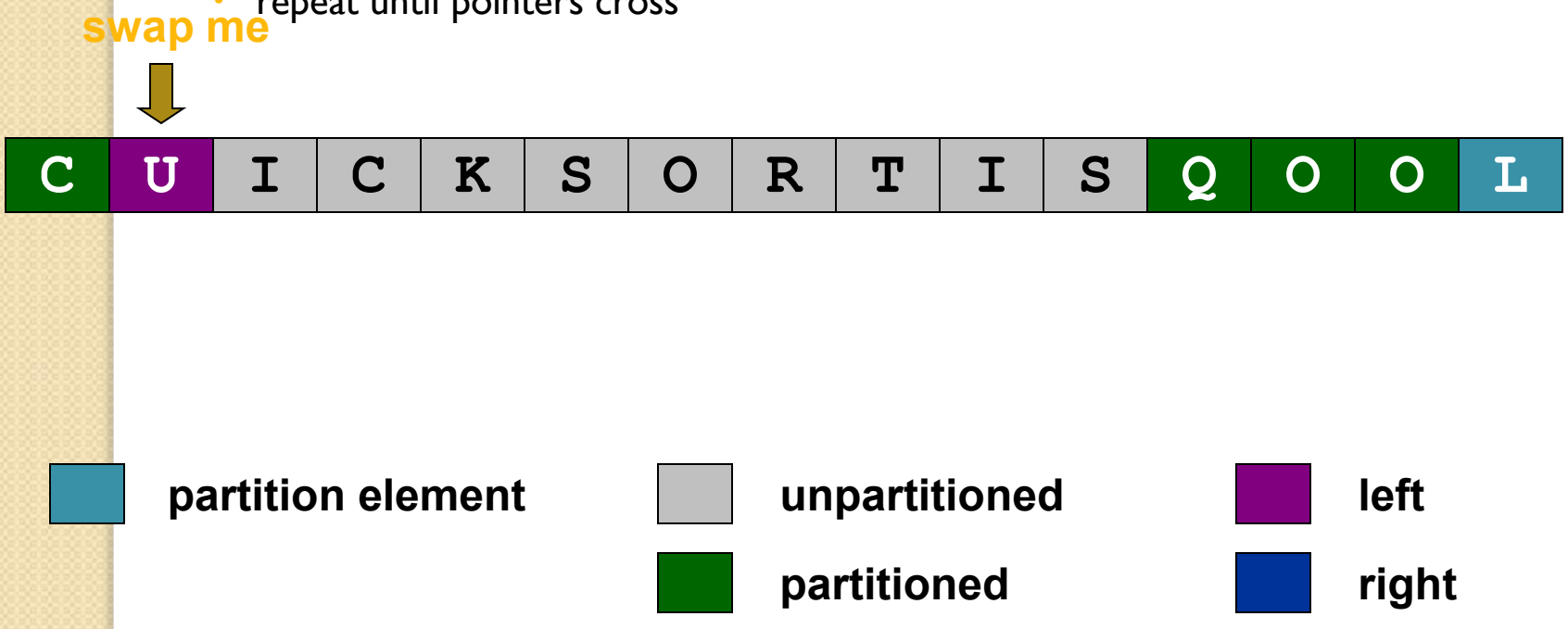
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



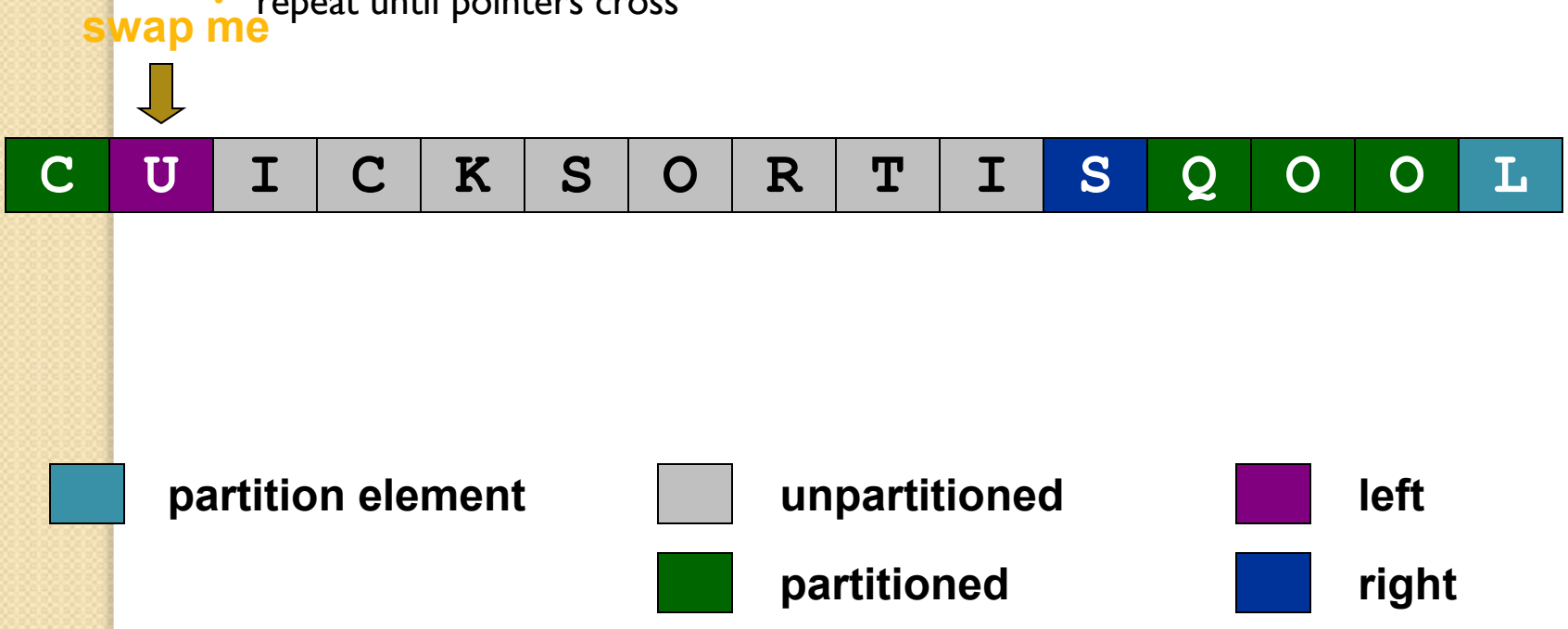
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



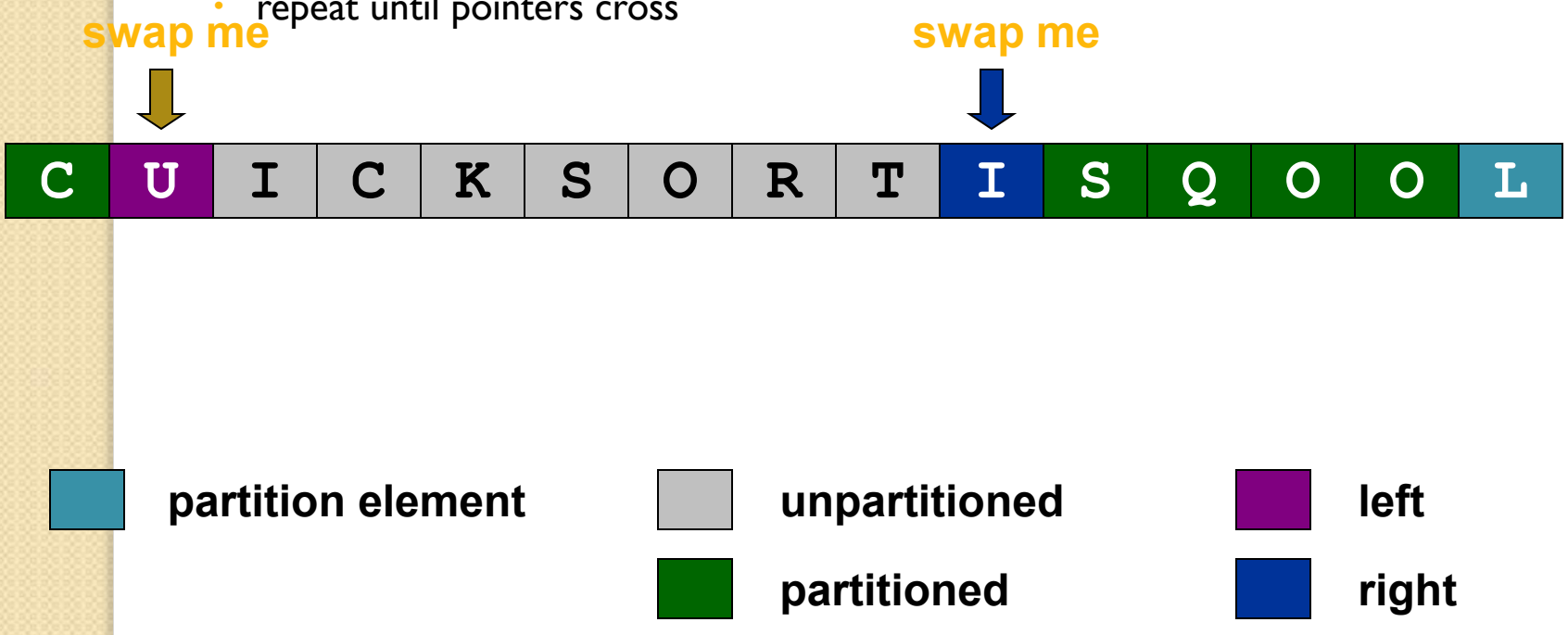
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



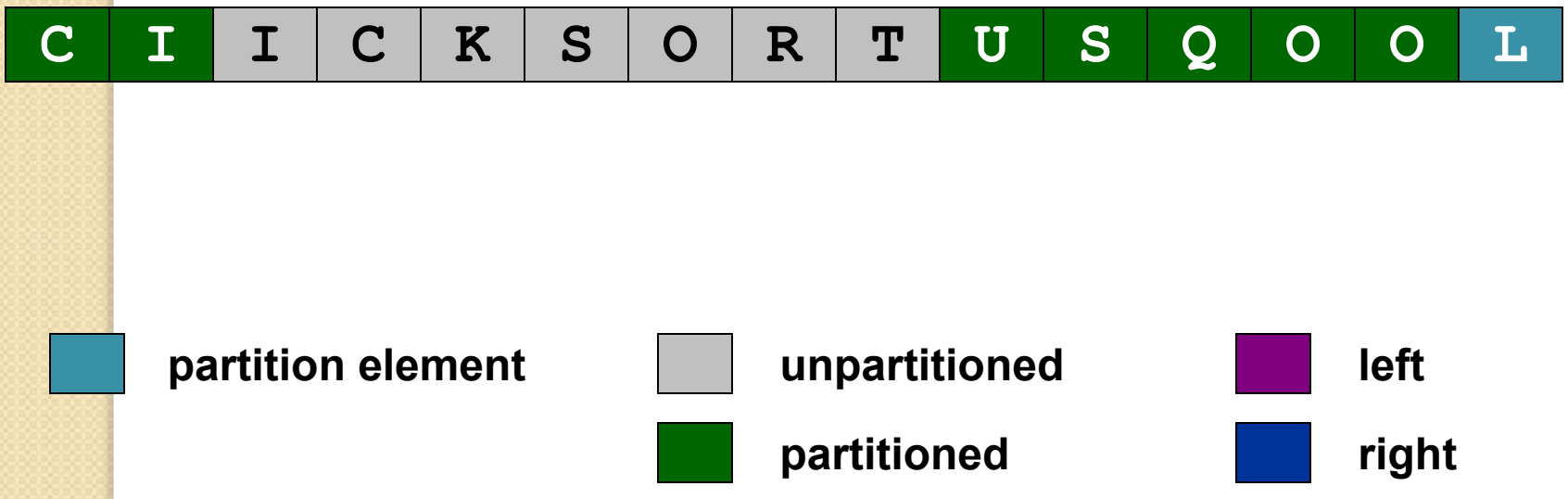
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



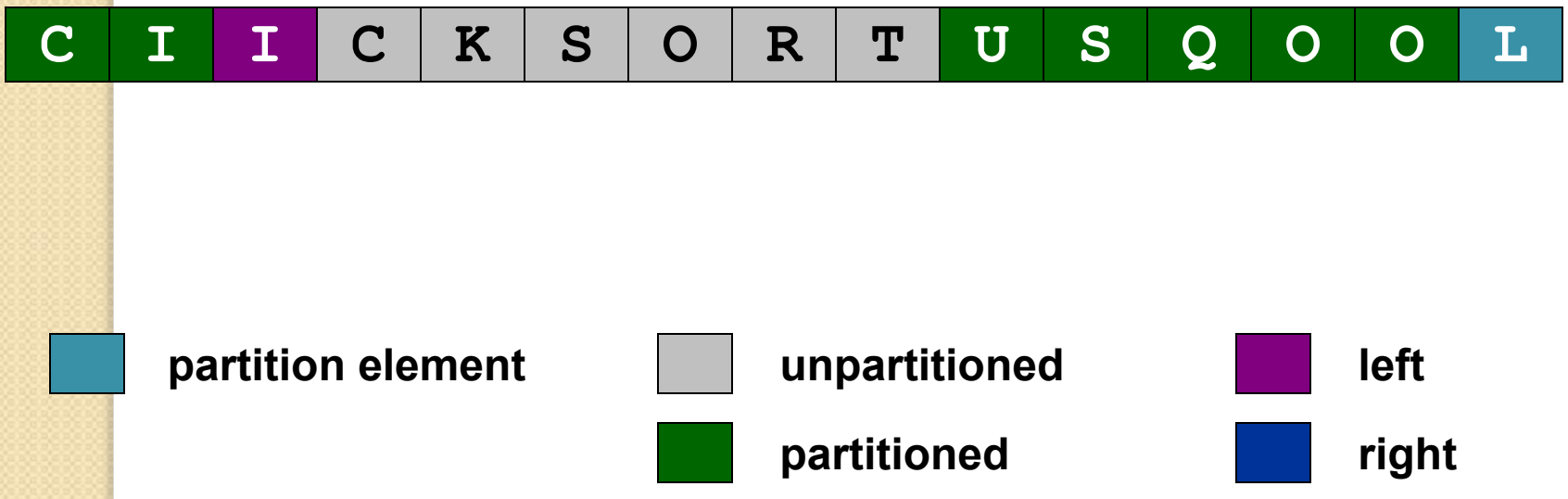
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



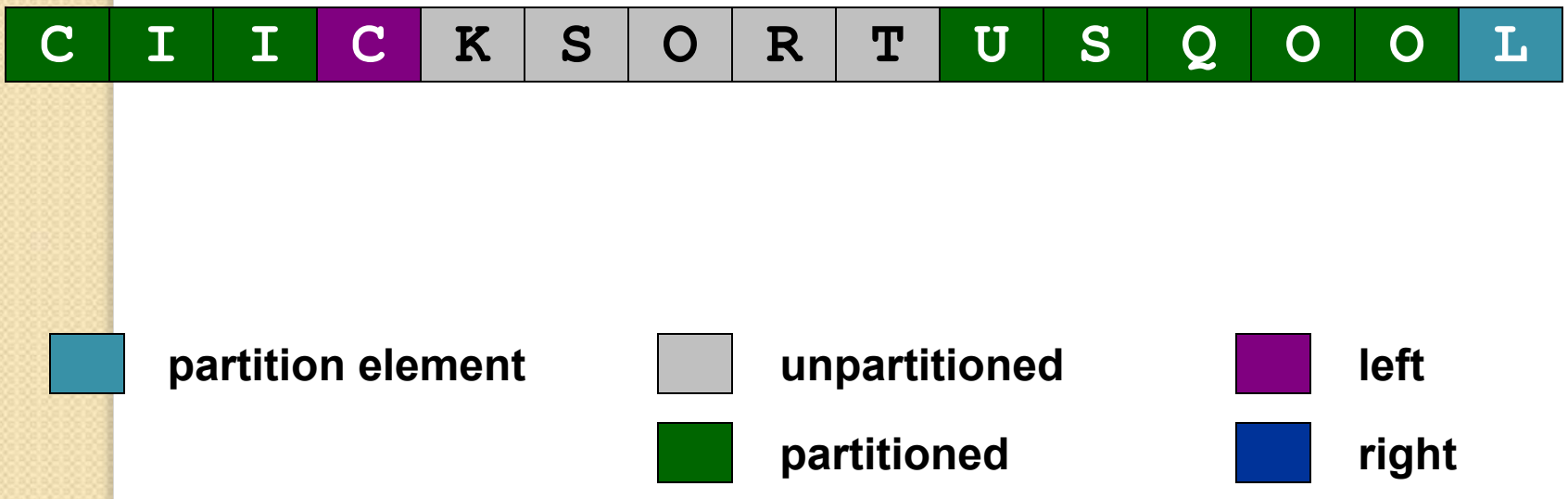
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



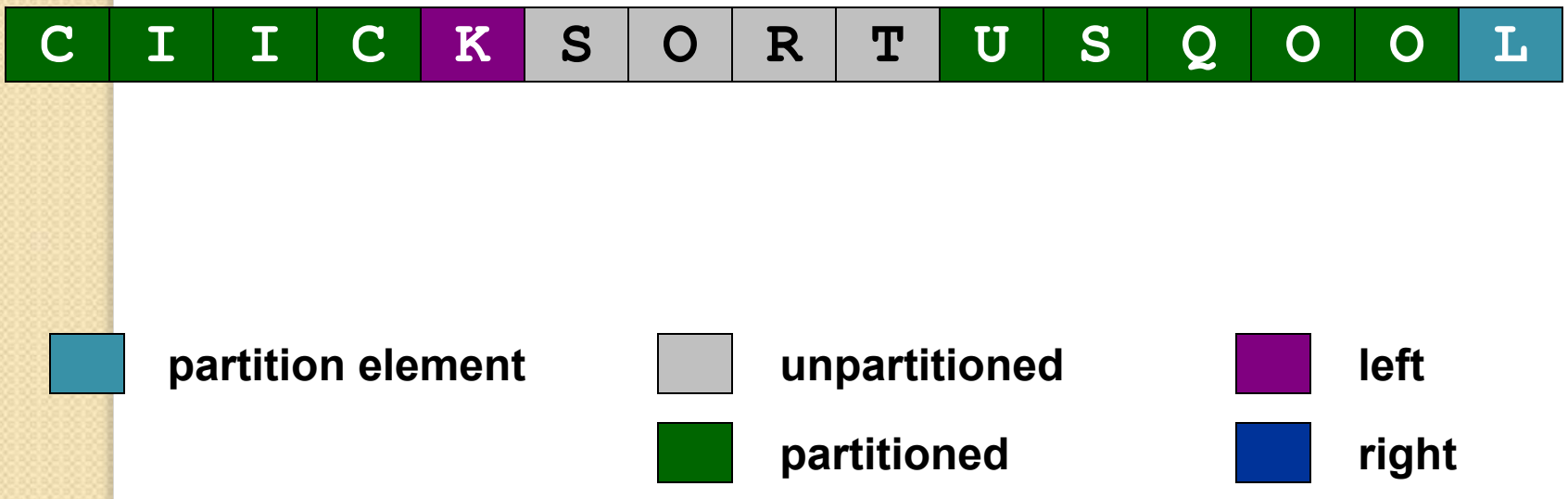
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - exchange
  - repeat until pointers cross



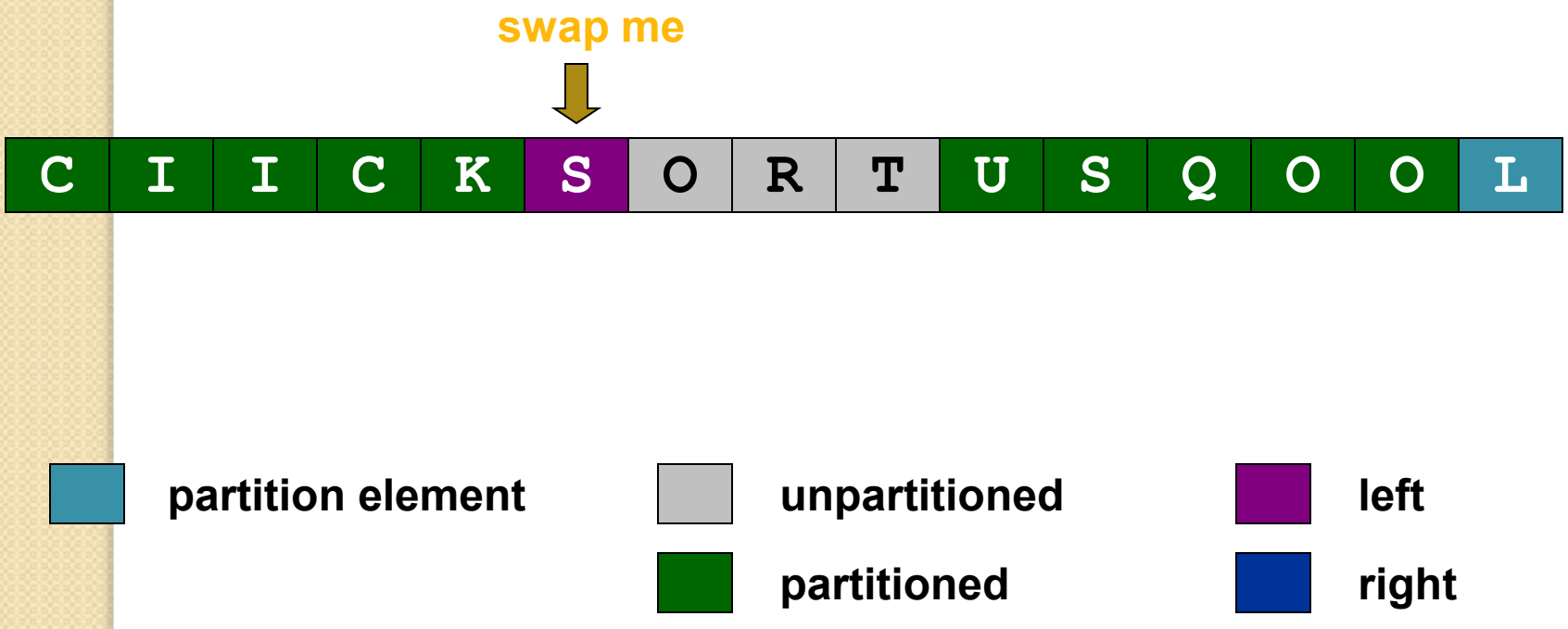
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross



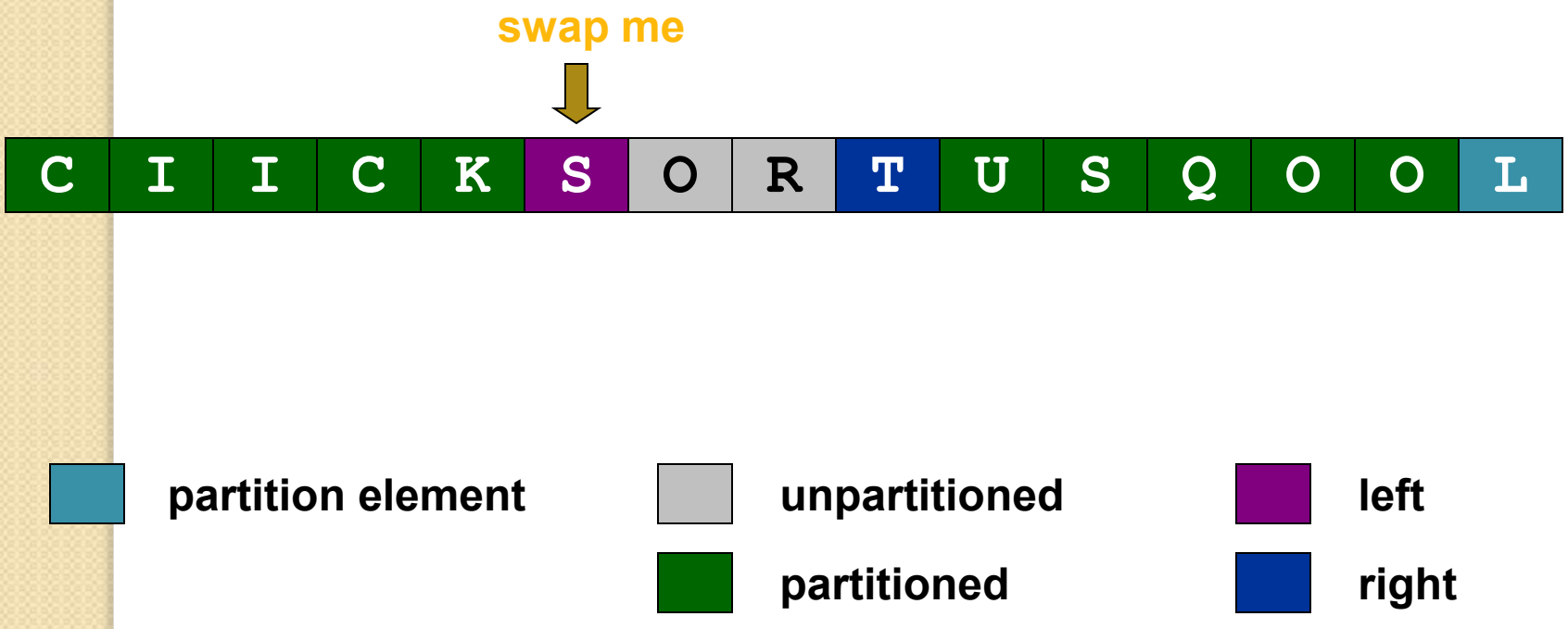
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross



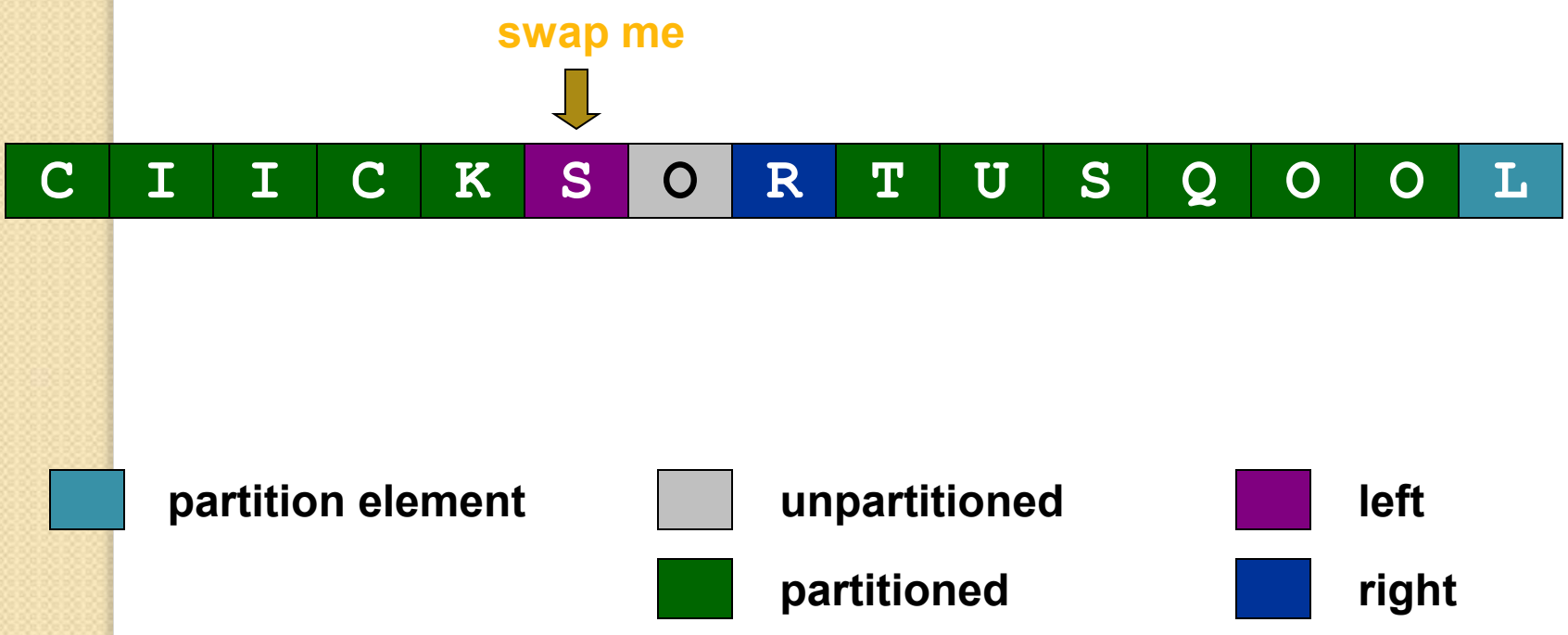
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross



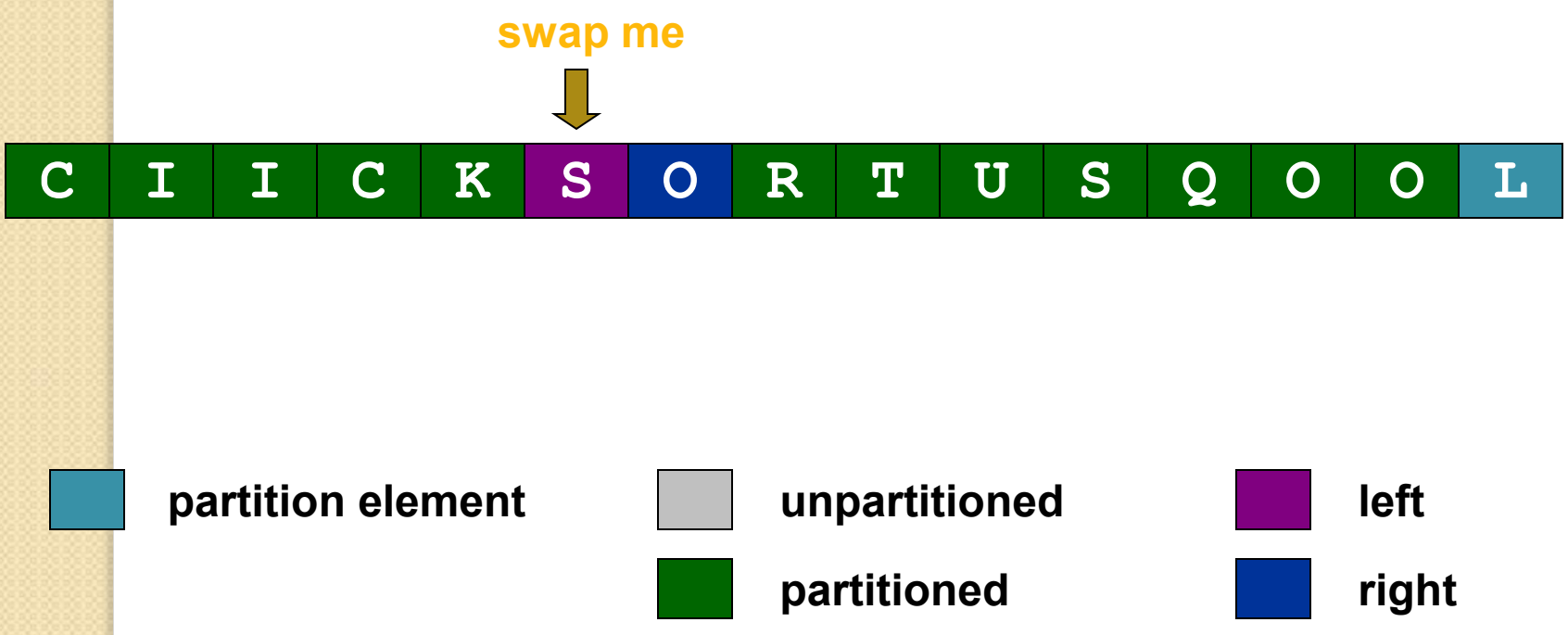
# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross



# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross

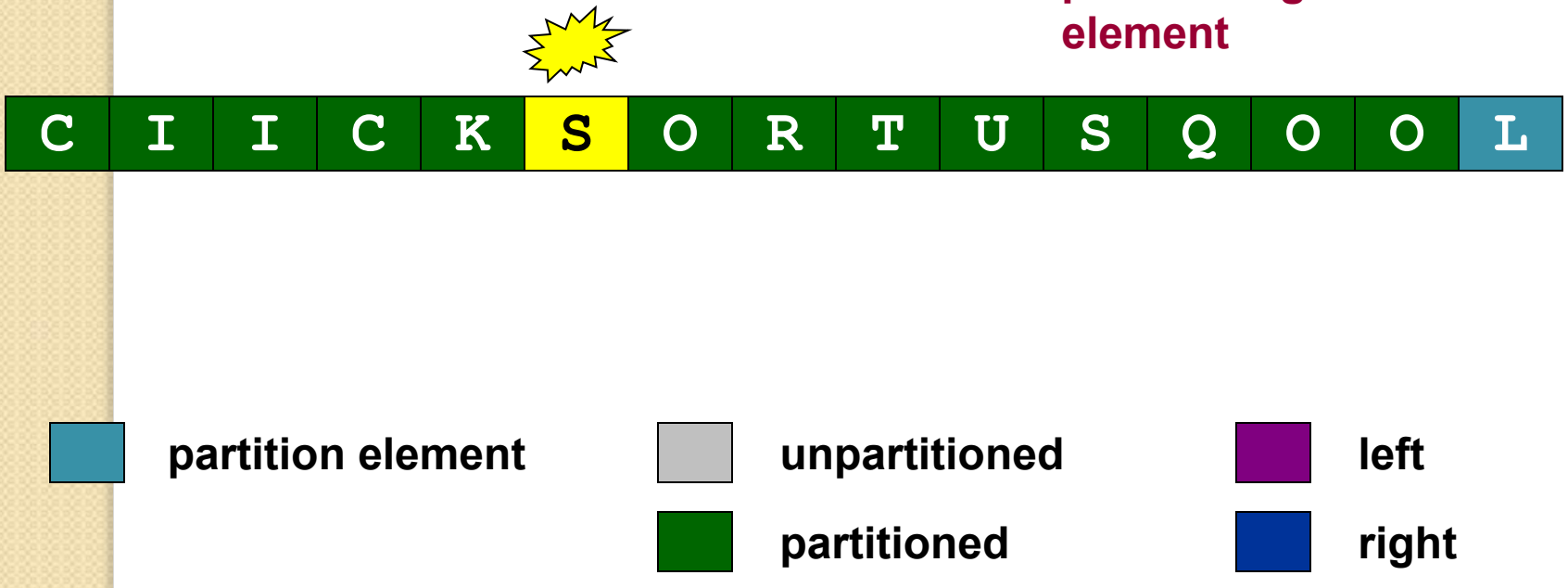


# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross

pointers cross

swap with partitioning element



# Partitioning in Quicksort

- How do we partition the array efficiently?
  - choose partition element to be rightmost element
  - scan from left for larger element
  - scan from right for smaller element
  - Exchange and repeat until pointers cross

**partition is complete**

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C | I | I | C | K | L | O | R | T | U | S | Q | O | O | S |

**partition element** **unpartitioned** **left**

**partitioned** **right**